PREDICTIVE ANALYTICS AND PERSONALIZED RECOMMENDATIONS FOR ENHANCED USER EXPERIENCE IN E-COMMERCE

A PROJECT REPORT

(NEXT BASKET)

Submitted By

Satheesh Kumar K

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

HINDUSTHAN INSTITUTE OF TECHNOLOGY, COIMBATORE

ANNA UNIVERSITY: CHENNAI 600 025

[MONTH YEAR]

BONAFIDE CERTIFICATE

Certified that this main project report "**Predictive Analytics and Personalized Recommendations for Enhanced User Experience in E-Commerce (Next Basket)**" is the bonafide work of "**Satheesh Kumar K**" who carried out the project work under my supervision.

SIGNATURE
Dr.M.Duraipandian, M.E,Ph.D.,
SUPERVISOR
Information Technology
Hindusthan Institute of Technology
Coimbatore - 641032

SIGNATURE
Dr.M.Duraipandian, M.E,Ph.D.,
HEAD OF THE DEPARTMENT
Information Technology
Hindusthan Institute of Technology
Coimbatore - 641032

Submitted for the University Project Viva Voice Examination Conducted on.....

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

I express my sincere thanks to Hindusthan Educational and Charitable Trust for providing me the necessary facilities to bring out the project successfully. I feel grateful to record my thanks to the Managing Trustee Tmt.Saraswathi Khannaiyann for all the support and the ray of strengthening hope extended to my project. It is a moment of immense pride for me to reveal profound thanks to our respected Principal, Dr.C.Natarajan, M.E.,Ph.D., who happens to be the striving force in all my endeavours.

I take immense pleasure in conveying my hearty thanks to Dr.M.Duraipandian, M.E.,Ph.D., Professor and Head of Information Technology Department for providing an opportunity to work on this project. His valuable suggestions helped me a lot to do this project successfully.

A word of thanks would not be sufficient for the work of my project guide Dr.M.Duraipandian, M.E.,Ph.D., Professor and Head of Information Technology Department who leads me through every trying circumstance. I deeply express my gratitude to all the Faculty Members and Support staff of the Department of Information Technology, for their encouragement, which I have received throughout my project.

Satheesh Kumar K

ABSTRACT

E-commerce is a business model that enables the buying and selling of goods and services over the internet. The global reach of the internet has led to the rise of numerous e-commerce platforms, all competing for a share of a growing digital market in which customer preferences and tastes change quickly. In this competitive environment, sellers struggle to predict demand and avoid stock shortages or overstocking, while buyers expect personalized and relevant product recommendations. Existing e-commerce systems often fall short in addressing these challenges, relying on manual monitoring, generic recommendations and limited automated alerts.

Next Basket is an integrated e-commerce system that addresses these problems through predictive analytics and a recommendation engine. The predictive analytics component uses an ARIMA (AutoRegressive Integrated Moving Average) model, trained on historical sales data, to forecast stock levels and to alert sellers about potential stock shortages. The recommendation engine uses a Decision Tree model, built on browsing behavior, purchase history and demographic information, to predict and suggest the next basket of items for buyers. The main objectives of the project are to provide sellers with proactive inventory management tools and to give buyers a personalized and engaging shopping experience, while keeping the system scalable and adaptable to changing market conditions and user behavior.

The system is developed as a web application using Python, the Flask framework, MySQL, HTML, CSS, JavaScript and Bootstrap, with TensorFlow, Pandas, Scikit-learn, NumPy and Matplotlib supporting the machine learning and data analysis tasks. It provides separate dashboards for the e-commerce administrator, sellers and customers, together with sales forecasting, next basket recommendation and SMS and e-mail notifications. The report presents the literature survey, system analysis, design, methodology, implementation, testing and results. In functional testing, 20 of the 22 test cases passed at the first attempt, and the remaining two minor issues were resolved. The project aims to contribute to the efficiency, profitability and user satisfaction of e-commerce platforms.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
1	INTRODUCTION	1
1.1.	OVERVIEW	1
1.1.1.	Types of E-commerce	2
1.2.	PROBLEM STATEMENT	3
1.3.	ARTIFICIAL INTELLIGENCE	4
1.3.1.	DECISION TREE	7
1.3.2.	ARIMA MODEL	9
1.4.	AIM AND OBJECTIVE	11
1.5.	SCOPE OF THE PROJECT	12
2	LITERATURE SURVEY	13
3	SYSTEM ANALYSIS	24
3.1.	EXISTING SYSTEM	24
3.1.1.	DISADVANTAGES	25
3.2.	PROPOSED SYSTEM	26
3.2.1.	ADVANTAGES	28
3.3.	FEASIBILITY STUDY	28
4	SYSTEM CONFIGURATION	30
4.1.	HARDWARE SPECIFICATIONS	30
4.2.	SOFTWARE SPECIFICATIONS	30
4.3.	ABOUT SOFTWARE	31
4.3.1.	PYTHON 3.7.4	31
4.3.2.	MYSQL	34
4.3.3.	WAMPSERVER	35
4.3.4.	BOOTSTRAP 4	35
4.3.5.	FLASK	36
4.3.6.	SMS API	37
4.3.7.	SIMPLE MAIL TRANSFER PROTOCOL	38
5	SYSTEM DESIGN	39
5.1.	SYSTEM ARCHITECTURE	39
5.2.	DATA FLOW DIAGRAM	40
5.2.1.	LEVEL 0	40
5.2.2.	LEVEL 1	41
5.2.3.	LEVEL 2	42
6	METHODOLOGY	43
6.1.	SYSTEM DESCRIPTION	43
6.2.	MODULES DESCRIPTION	46
6.2.1.	E-Commerce Web App	46
6.2.2.	Forecaster Model	48
6.2.3.	Recommendation Engine (Decision Tree): Build and Train	50

6.2.4. Sales Forecasting 51
6.2.5. Next Basket Recommendation 52
6.2.6. Actors Dashboard 53
6.2.7. Notification 55
6.3. SYSTEM IMPLEMENTATION 55
6.4. SYSTEM TESTING 57
6.5. TEST CASES 59
6.6. TEST REPORT 63
7 RESULTS 65
7.1. SCREENSHOTS 65
7.2. PACKAGES 71
7.3. DATABASE DESIGN 76
8 CONCLUSION 80
REFERENCES 82

LIST OF FIGURES

Figure 1: Overview of E-commerce 1
Figure 2: Types of E-commerce 2
Figure 3: Artificial Intelligence Terms 5
Figure 4: Structure of a Decision Tree 8
Figure 5: ARIMA Model Equation 10
Figure 6: Components of the ARIMA Model 10
Figure 7: Python 31
Figure 8: TensorFlow 32
Figure 9: Pandas 32
Figure 10: NumPy 33
Figure 11: Matplotlib 33
Figure 12: Scikit-learn 33
Figure 13: MySQL 34
Figure 14: WampServer 35
Figure 15: Bootstrap 4 35
Figure 16: Flask 36
Figure 17: SMS API 37
Figure 18: Simple Mail Transfer Protocol (SMTP) 38
Figure 19: System Architecture 39
Figure 20: Level 0 Data Flow Diagram 40
Figure 21: Level 1 Data Flow Diagram 41
Figure 22: Level 2 Data Flow Diagram 42
Figure 23: Admin Login 65
Figure 24: Training Phase 65
Figure 25: Forecaster Model: Data Collection 66
Figure 26: Forecaster Model: Pre-processing 66
Figure 27: Forecaster Model: Data Visualization 66
Figure 28: Forecaster Model: Build and Train 67
Figure 29: Forecaster Model: Performance Analysis 67
Figure 30: Recommendation Engine: Data Collection 67
Figure 31: Recommendation Engine: Pre-processing 68
Figure 32: Recommendation Engine: Feature Selection 68

Figure 33: Recommendation Engine: Feature Extraction 68
Figure 34: Recommendation Engine: Model Build and Performance Analysis 69
Figure 35: Seller Login 69
Figure 36: Add Product 69
Figure 37: Customer Registration 70
Figure 38: Customer Login 70
Figure 39: Products 70
Figure 40: Cart and Next Basket Recommendation 71
Figure 41: Purchased Orders 71
Figure 42: Sales Forecasting 71

LIST OF TABLES

Table 1: Bug Report.....64
Table 2: Structure of the Admin Table 76
Table 3: Structure of the Training Table.....77
Table 4: Structure of the Category Table.....77
Table 5: Structure of the Product Table.....77
Table 6: Structure of the Register Table 77
Table 7: Structure of the User Searching Table.....78
Table 8: Structure of the Product Recommendation Table.....78
Table 9: Structure of the Shopping Card Table 78
Table 10: Structure of the Payment Table 78
Table 11: Structure of the User Review Table.....79

LIST OF ABBREVIATIONS

AI	-	Artificial Intelligence
API	-	Application Programming Interface
ARIMA	-	AutoRegressive Integrated Moving Average
CSS	-	Cascading Style Sheets
GUI	-	Graphical User Interface
HTML	-	HyperText Markup Language
IDE	-	Integrated Development Environment
ML	-	Machine Learning
RDBMS	-	Relational Database Management System
SMS	-	Short Message Service
SMTP	-	Simple Mail Transfer Protocol

1 INTRODUCTION

1.1. OVERVIEW

Ecommerce = electronic commerce. In simple terms, it means running an online business. It refers to the selling and purchasing of goods and services over the internet where the exchange of money and data takes place via protected connections in order to execute a transaction. Ecommerce offers a number of benefits over “brick and mortar” stores. It allows customers to easily find their desired product(s) through a large database. They can research the product beforehand, compare prices, learn more from customer reviews, and then buy it once their concerns are rectified by the online customer support representative.

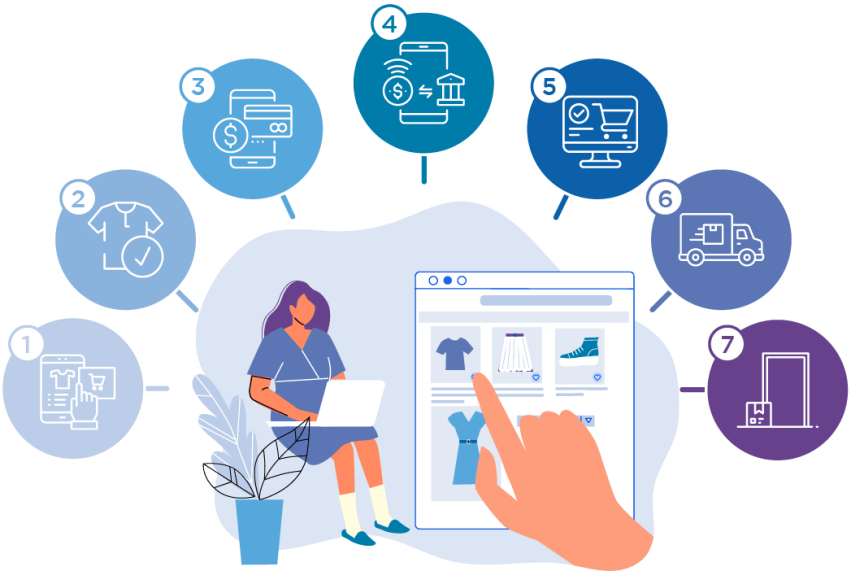


Figure 1: Overview of E-commerce

E-commerce has been around since the early 1990s when Amazon just sold books. Today, it’s a multibillion-dollar industry. According to Statista, the U.S. e-commerce industry is expected to eclipse $1 trillion in value in 2024, making it a major economic staple for the national economy. E-commerce is an easy-to-launch and suitable-for-most business idea, which has gained the attention of entrepreneurs of all ages in the country. From local shop owners to huge organizations, everyone is looking to create a presence in the online business world. The future of e-commerce can be considered to be bright, and the best time to enter the domain is now. E-commerce works in the same way as any physical store. The only difference is that e-commerce is powered by the internet and is also not limited to any geographical area or region.

1.1.1. Types of E-commerce

Depending on the goods, services, and organization of an ecommerce company, the business can opt to operate several different ways. Here are several of the popular business models.



Figure 2: Types of E-commerce

- **Business-to-Consumer (B2C)**

B2C e-commerce companies sell directly to the product end-user. Instead of distributing goods to an intermediary, a B2C company performs transactions with the consumer that will ultimately use the good. This type of business model may be used to sell products (like your local sporting goods store's website) or services (such as a lawn care mobile app to reserve landscaping services). This is the most common business model and is likely the concept most people think about when they hear the term e-commerce.

- **Business-to-Business (B2B)**

Similar to B2C, an e-commerce business can directly sell goods to a user. However, instead of being a consumer, that user may be another company. B2B transactions often entail larger quantities, greater specifications, and longer lead times. The company placing the order may also have a need to set recurring goods if the purchase is for recurring manufacturing processes.

- **Business-to-Government (B2G)**

Some entities specialize as government contractors providing goods or services to agencies or administrations. Similar to a B2B relationship, the business produces items of value and remits those items to an entity. B2G e-commerce companies must often meet government requests for proposal requirements, solicit bids for projects, and meet very specific product or service criteria. In addition, there may be joint government endeavors to solicit a single contract through a government-wide acquisition contract.

- **Consumer-to-Consumer (C2C)**

Established companies are the only entities that can sell things. E-commerce platforms such as digital marketplaces connect consumers with other consumers who can list their own products and execute their own sales. These C2C platforms may be auction-style listings (i.e. eBay auctions) or may warrant further discussion regarding the item or service being provided (i.e. Craigslist postings). Enabled by technology, C2C e-commerce platforms empower consumers to both buy and sell without the need for companies.

- **Consumer-to-Business (C2B)**

Modern platforms have allowed consumers to more easily engage with companies and offer their services, especially related to short-term contracts, gigs, or freelance opportunities. For example, consider listings on Upwork. A consumer may solicit bids or interact with companies that need particular jobs done. In this way, the e-commerce platform connects businesses with freelancers to enable consumers greater power to achieve pricing, scheduling, and employment demands.

- **Consumer-to-Government (C2G)**

Less of a traditional e-commerce relationship, consumers can interact with administrations, agencies, or governments through C2G partnerships. These partnerships are often not in the exchange of service but rather, the transaction of obligation. For example, uploading your federal tax return to the Internal Revenue Service (IRS) digital website is an e-commerce transaction regarding an exchange of information. Alternatively, you may pay your tuition to your university online or remit property tax assessments to your county assessor.

1.2. PROBLEM STATEMENT

E-commerce businesses face significant challenges in managing their stock effectively, primarily stemming from difficulties in accurately predicting demand and inefficient inventory

management processes. The task of forecasting demand is complex, influenced by factors such as seasonality, shifting consumer trends, and external market dynamics. Inaccurate predictions often lead to stockouts, where popular items become unavailable, resulting in lost sales opportunities and disgruntled customers. Conversely, overestimating demand leads to overstocking, tying up capital in excess inventory and increasing storage costs. Additionally, outdated inventory management processes contribute to inefficiencies in tracking stock levels, replenishing inventory, and managing stock across multiple channels or warehouses. Manual or outdated systems lack real-time visibility into inventory movements, hindering decision-making and resulting in suboptimal inventory allocation and fulfillment processes. These inefficiencies drive up operational costs, including storage expenses and labor costs, while also increasing the risk of inventory obsolescence.

Furthermore, e-commerce platforms often struggle with personalization limitations, depriving users of a tailored shopping experience. Without personalized product recommendations, platforms deliver generic shopping experiences where users are presented with a broad range of products that may not align with their preferences or interests. This generic approach undermines engagement, reduces conversion rates, and can lead to missed revenue opportunities. Moreover, failure to tailor recommendations based on individual preferences and past purchase behavior diminishes customer satisfaction and loyalty. In today's competitive landscape, customers expect personalized experiences that cater to their unique preferences and needs. The absence of personalized recommendations also hinders cross-selling and up-selling opportunities, as platforms miss chances to suggest complementary or higher-value products based on customers' purchase histories or browsing behaviors. To address these challenges, e-commerce businesses must invest in advanced analytics and machine learning algorithms to improve demand forecasting accuracy and deliver personalized product recommendations. By leveraging data-driven insights and automation technologies, businesses can optimize their inventory management processes and enhance the overall shopping experience for customers.

1.3. ARTIFICIAL INTELLIGENCE

An AI is a computer system that is able to perform tasks that ordinarily require human intelligence. These artificial intelligence systems are powered by machine learning. Many of them are powered by machine learning, some of them are powered by specifically deep learning, some of them are powered by very boring things like just rules. Artificial intelligence

systems are critical for companies that wish to extract value from data by automating and optimizing processes or producing actionable insights. Artificial intelligence systems powered by machine learning enable companies to leverage large amounts of available data to uncover insights and patterns that would be impossible for any one person to identify, enabling them to deliver more targeted, personalized communications, predict critical care events, identify likely fraudulent transactions, and more.

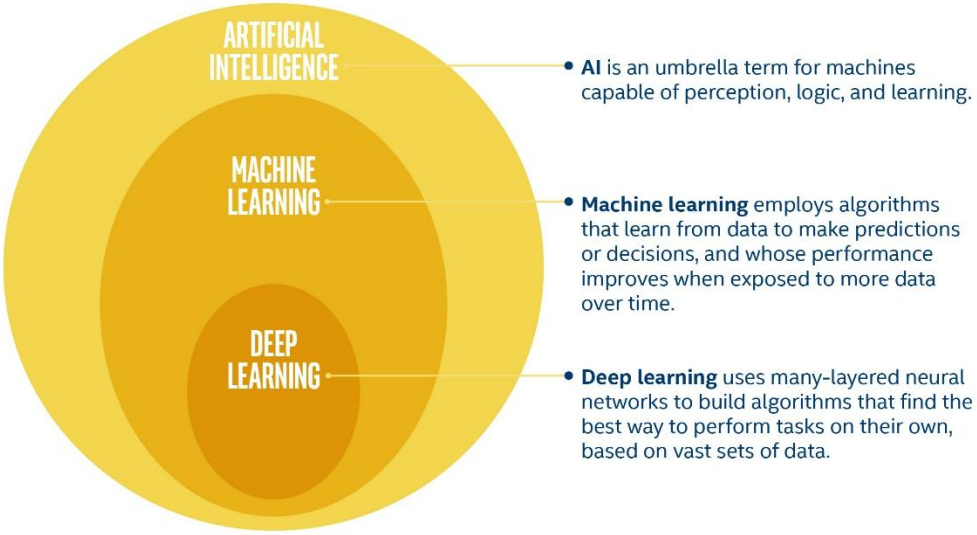


Figure 3: Artificial Intelligence Terms

Artificial intelligence systems work by using any number of AI techniques.

Machine Learning

A machine learning (ML) algorithm is fed data by a computer and uses statistical techniques to help it "learn" how to get progressively better at a task, without necessarily having been programmed for that certain task. It uses historical data as input to predict new output values. Machine learning consists of both supervised learning (where the expected output for the input is known thanks to labeled data sets) and unsupervised learning (where the expected outputs are unknown due to the use of unlabeled data sets).

Deep Learning

Deep learning is a type of machine learning that runs inputs through a biologically inspired neural network architecture. The neural networks contain a number of hidden layers through which the data is processed, allowing the machine to go "deep" in its learning, making connections and weighting input for the best results.

The Future Impact of Artificial Intelligence on the Global Supply Chain

The global supply chain is a complex network of interconnected processes that enable the efficient flow of goods and services around the world. As technology continues to advance at an unprecedented pace, one innovation stands out as a game-changer: Artificial Intelligence (AI). AI has the potential to revolutionize the global supply chain, enhancing efficiency, optimizing decision-making, and driving significant cost savings. In this article, we will explore the transformative power of AI and how it will reshape the global supply chain in the future.

- **Enhanced Demand Forecasting**

Accurate demand forecasting is crucial for supply chain management. AI algorithms can analyze vast amounts of historical and real-time data, considering various factors such as customer behavior, market trends, and even external factors like weather patterns. By leveraging this data, AI-powered systems can provide more accurate demand forecasts, enabling companies to optimize inventory levels, reduce stockouts, and improve overall supply chain responsiveness.

- **Intelligent Inventory Management**

Optimizing inventory management is a perennial challenge for supply chain professionals. AI can provide real-time insights into inventory levels, consumption patterns, and lead times, enabling businesses to achieve leaner operations and reduce costs. By employing machine learning algorithms, AI systems can predict optimal inventory levels, minimize excess inventory, and even automate replenishment processes. Such intelligence-driven inventory management will foster better demand fulfillment, reduce waste, and improve cash flow.

- **Smart Warehousing and Robotics**

AI-driven technologies are transforming the warehousing landscape. Intelligent robots equipped with computer vision and machine learning capabilities can enhance warehouse operations, automating tasks such as sorting, picking, and packing. These robots can work alongside human employees, increasing efficiency and reducing errors. AI-powered warehouse management systems can optimize layout, streamline routing, and improve resource allocation, resulting in faster order fulfillment and increased customer satisfaction.

- **Predictive Maintenance**

Equipment breakdowns and maintenance issues can disrupt the supply chain and cause significant financial losses. AI can mitigate such risks by implementing predictive maintenance strategies. By analyzing real-time sensor data, AI algorithms can identify patterns, detect anomalies, and predict equipment failures before they occur. This proactive approach allows companies to schedule maintenance activities, minimize downtime, and reduce unplanned repairs, resulting in improved operational efficiency and cost savings.

- **Supply Chain Visibility and Traceability**

Global supply chains often suffer from a lack of visibility and transparency. AI-powered systems can overcome this challenge by enabling real-time tracking and traceability across the supply chain. By leveraging technologies such as Internet of Things (IoT) devices, RFID tags, and blockchain, AI can provide end-to-end visibility, from raw material sourcing to final product delivery. This increased transparency enhances accountability, reduces counterfeit risks, and ensures compliance with regulatory standards.

- **Optimized Route Planning and Logistics**

AI algorithms can optimize route planning and logistics operations, considering various factors such as traffic conditions, fuel costs, and delivery time windows. By leveraging historical and real-time data, AI can identify the most efficient routes, recommend alternative transportation modes, and even optimize vehicle loading. These intelligent logistics systems can reduce transportation costs, minimize carbon emissions, and enhance overall supply chain agility.

- **Supplier Relationship Management**

Maintaining strong relationships with suppliers is crucial for a resilient supply chain. AI can facilitate supplier relationship management by automating routine communication, streamlining order processing, and even predicting supplier performance. By analyzing supplier data and external market factors, AI-powered systems can identify potential risks and opportunities, allowing businesses to make informed decisions and build robust supplier networks.

1.3.1. DECISION TREE

A decision tree is a map of the possible outcomes of a series of related choices. It allows an individual or organization to weigh possible actions against one another based on their costs,

probabilities, and benefits. As the name goes, it uses a tree-like model of decisions. They can be used either to drive informal discussion or to map out an algorithm that predicts the best choice mathematically. A decision tree typically starts with a single node, which branches into possible outcomes. Each of those outcomes leads to additional nodes, which branch off into other possibilities. This gives it a tree-like shape.

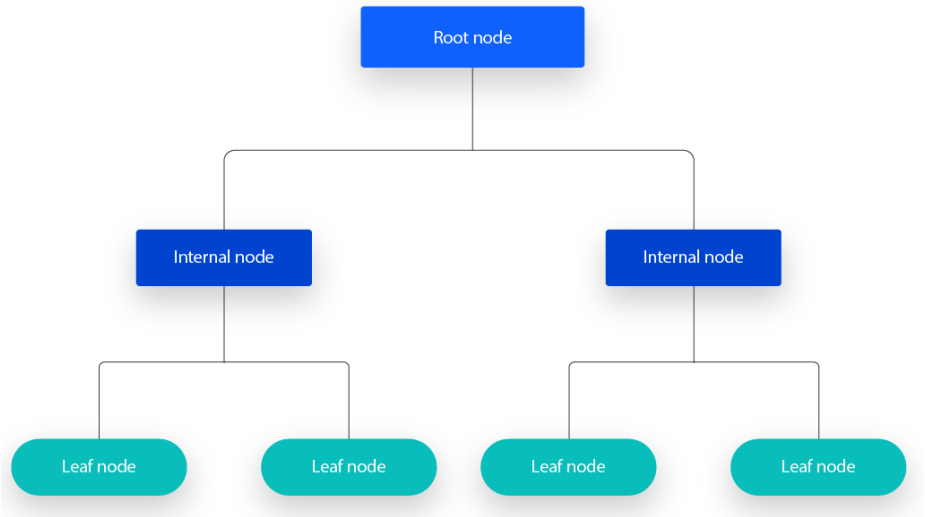


Figure 4: Structure of a Decision Tree

There are three different types of nodes: chance nodes, decision nodes, and end nodes. A chance node, represented by a circle, shows the probabilities of certain results. A decision node, represented by a square, shows a decision to be made, and an end node shows the final outcome of a decision path. Decision tree learning employs a divide and conquer strategy by conducting a greedy search to identify the optimal split points within a tree. This process of splitting is then repeated in a top-down, recursive manner until all, or the majority of records have been classified under specific class labels. Whether or not all data points are classified as homogenous sets is largely dependent on the complexity of the decision tree. Smaller trees are more easily able to attain pure leaf nodes—i.e. data points in a single class. However, as a tree grows in size, it becomes increasingly difficult to maintain this purity, and it usually results in too little data falling within a given subtree. When this occurs, it is known as data fragmentation, and it can often lead to overfitting. As a result, decision trees have preference for small trees, which is consistent with the principle of parsimony in Occam's Razor; that is, "entities should not be multiplied beyond necessity." Said differently, decision trees should add complexity only if necessary, as the simplest explanation is often the best. To reduce complexity and prevent overfitting, pruning is usually employed; this is a process, which removes branches that split on features with low importance. The model's fit can then be

evaluated through the process of cross-validation. Another way that decision trees can maintain their accuracy is by forming an ensemble via a random forest algorithm; this classifier predicts more accurate results, particularly when the individual trees are uncorrelated with each other.

The Greedy Approach

"Greedy Approach is based on the concept of Heuristic Problem Solving by making an optimal local choice at each node. By making these local optimal choices, we reach the approximate optimal solution globally."

The algorithm can be summarized as:

1. At each stage (node), pick out the best feature as the test condition.
2. Now split the node into the possible outcomes (internal nodes).
3. Repeat the above steps till all the test conditions have been exhausted into leaf nodes.

When you start to implement the algorithm, the first question is: *'How to pick the starting test condition?'*

The answer to this question lies in the values of *'Entropy*' and *'Information Gain*'. Let us see what are they and how do they impact our decision tree creation.

Entropy: Entropy in Decision Tree stands for homogeneity. If the data is completely homogenous, the entropy is 0, else if the data is divided (50-50%) entropy is 1.

Information Gain: Information Gain is the decrease/increase in Entropy value when the node is split.

An attribute should have the highest information gain to be selected for splitting. Based on the computed values of Entropy and Information Gain, we choose the best attribute at any particular step.

1.3.2. ARIMA MODEL

ARIMA is the abbreviation for Auto Regressive Integrated Moving Average. Auto Regressive (AR) terms refer to the lags of the differenced series, Moving Average (MA) terms refer to the lags of errors and I is the number of difference used to make the time series stationary. ARIMA is also known as the Box-Jenkins approach. Box and Jenkins claimed that non-stationary data can be made stationary by differencing the series, Yt. The general model for Yt is written as,

$$Y_t = \phi 1 Y_t - 1 + \phi 2 Y_t - 2 \ldots \phi p Y_t - p + \epsilon_t + \theta 1 \epsilon_t - 1 + \theta 2 \epsilon_t - 2 + \ldots \theta q \epsilon_t - q$$

Figure 5: ARIMA Model Equation

Where Yt is the differenced time series value, ϕ and θ are unknown parameters and ϵ are independent identically distributed error terms with zero mean.

Here, Yt is expressed in terms of its past values and the current and past values of error terms.

This model is called Autoregressive Integrated Moving Average or ARIMA(p,d,q) of Yt. This is also termed as the ARIMA equation.

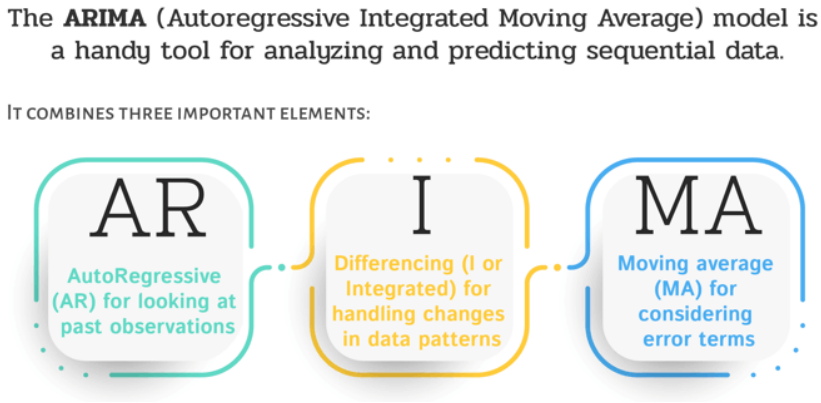


Figure 6: Components of the ARIMA Model

Forecasting Model in Time Series

Forecasting involves predicting future values based on historical data, either for a variable itself or by examining the relationship between variables. There are two main categories of forecasting approaches: qualitative and quantitative. In quantitative forecasting, we focus on time series data and apply statistical principles to predict future values.

Some popular techniques for time series forecasting include:

- **Autoregressive Models (AR)**

These models assume that future values of a variable depend on its past values. By looking at the patterns and relationships within the variable's historical data, AR models can estimate future values.

- **Moving Average Models (MA)**

MA models, on the other hand, consider the average of past error terms to forecast future values. They capture the random fluctuations or noise in the data.

- **Seasonal Regression Models**

These models incorporate seasonal patterns into the forecasting process. They take into account variables that represent the seasonal effects observed in the data, helping us make predictions with periodicity in mind.

- **Distributed Lags Models**

Distributed lags models analyses the relationship between a variable's current value and its lagged values. By considering the impact of past values on present and future values, these models offer insights into forecasting.

1.4. AIM AND OBJECTIVE

AIM

The aim of the project is to revolutionize e-commerce operations by developing an integrated system that utilizes advanced predictive analytics and recommendation models. Specifically, the system aims to address the challenges faced by both sellers and buyers in the e-commerce domain.

OBJECTIVES

- To develop an ARIMA model capable of accurately predicting and forecasting stock levels for sellers.
- To implement mechanisms within the predictive analytics component to alert sellers promptly about potential stock shortages based on the ARIMA forecasts.
- To utilize a Decision Tree model within the recommendation engine to predict and suggest the next basket of items for buyers, leveraging their preferences and past purchase behavior.
- To provide proactive tools for sellers aimed at enhancing inventory management efficiency, thereby minimizing stock shortages and optimizing stock levels.
- To deliver a highly personalized shopping experience for buyers by offering relevant product recommendations tailored to their individual preferences.
- To design and implement a scalable system architecture integrating machine learning techniques, data analytics, and real-time integration capabilities.
- To ensure adaptability of the system to changing market conditions and evolving user behaviors, facilitating seamless operation and maintenance.

- To contribute to the efficiency and profitability of e-commerce platforms by optimizing inventory management, increasing sales opportunities, and enhancing overall user satisfaction.
- To revolutionize e-commerce operations by offering innovative solutions that address common challenges faced by both sellers and buyers.
- To significantly enhance user satisfaction within the e-commerce domain by providing proactive inventory management tools for sellers and personalized shopping experiences for buyers.

1.5. SCOPE OF THE PROJECT

The scope of this project encompasses the development and integration of advanced predictive analytics and recommendation models within e-commerce platforms. This includes the implementation of an ARIMA model to accurately predict and forecast stock levels for sellers, along with mechanisms to promptly alert them about potential stock shortages. Additionally, a Decision Tree model will be utilized to predict and suggest the next basket of items for buyers, offering personalized recommendations based on their preferences and past purchase behavior. The project involves providing proactive inventory management tools for sellers, such as features for stock monitoring, forecasting, and alerting, as well as delivering a highly personalized shopping experience for buyers through relevant product recommendations. A key aspect of the project is the design and implementation of a scalable system architecture capable of handling large volumes of data and supporting real-time integration with existing e-commerce platforms. User interfaces will be designed to be intuitive and user-friendly, catering to the needs of both sellers and buyers. Thorough testing, documentation, and training materials will be provided to ensure the system's accuracy, reliability, and usability, with ongoing support and maintenance services offered to address any issues and ensure continued functionality and relevance.

2 LITERATURE SURVEY

1. **Recommendation System with Hierarchical Recurrent Neural Network for Long-Term Time Series**

   Author: Byeongjin Choe; Taegwan Kang; Kyomin Jung

   Year: 2021

   Doi: https://ieeexplore.ieee.org/document/9430555

Objective

The objective of this project is to predict what item a user will use next using large amounts of stored user history.

Methodology

In this study, the author presents an RNN-based hierarchical model to consider both long sequence lengths and time interval information. A typical RNN-based model struggles to differentiate between events that occur at a certain interval of time and events that occur irregularly. This model applied a hierarchical structure to effectively use time information to handle different time intervals differently in the model. This model has two layers: a short-term layer and a long-term layer. The short-term layer deals with short-term sequences that were commonly dealt with in previous models. The long-term layers remember information from previous short-term sequences and deliver it over a long time considering the time interval.

Findings

This study has high performance compared with several baseline models from experiments. The experiments indicates that it is important to use temporal properties in long-term time series datasets. construct the hierarchical model based on RNN that is effective in a long-term time series.

2. **Improving Explainable Recommendations by Deep Review-Based Explanations**

   Author: Sixun Ouyang; Aonghus Lawlor

   Year: 2021

   Doi: https://ieeexplore.ieee.org/document/9417205

Objective

The objective of the paper is to develop two character-level deep neural network-based personalised review generation models, and improve recommendation accuracy by generating high-quality text which meets the input criteria of text-aware recommender systems. To make fair comparisons, the author train review-aware recommender systems by human written reviews and attain advanced recommendations by feeding generated reviews at the inference step.

Methodology

In this paper the author proposes deep learning review generation models for high-quality natural language explanations and the author use these explanations to reach state of the art performance on rating prediction and top-N item ranking recommendation tasks. Specifically, the author develops two character-level personalised review generation models apply to generate useful reviews for recommendation and explanation. y Long Short-Term Memory (LSTM) neural network as the basis for our generation models and conduct cross-domain experiments on two Convolutional Neural Network (CNN) based recommender systems.

Findings

Improved recommendation accuracy by inputting compelling machine-generated review-based explanations. Experimental results revealed that this models consistently surpass the baselines and achieve state-of-the-art rating prediction performance. In ranking evaluation, the attention generation model outperformed baselines and showed excellent performance. The author showed that generation models are suitable ingredients for recommender systems since they focus on generating useful machine-generated reviews.

3. **Selective Knowledge Transfer for Cross-Domain Collaborative Recommendation**

   Author: Hongwei Zhang; Xiangwei Kong; Yujia Zhang

   Year: 2021

   Doi: https://ieeexplore.ieee.org/document/9360540

Objective

The main aim is to help boost the prediction performance of the target domain with numerical ratings (e.g., 5-star rating) by using auxiliary domain with binary ratings (e.g., likes or dislikes) that overlap with the items in the target domain.

Methodology

Selective Know Transfer for Cross-domain Collaborative Recommendation (SKT). First, SKT jointly factorizes the auxiliary rating matrix and the target rating matrix with the constraint of sharing the same latent factor of item and selectively sharing the latent factor of user. Second, to ensure positive transfer, the author integrate the graph co-regularization of user graph and item graph into the proposed SKT model to maintain the respective intrinsic geometric property of the learned latent factor. extend the CMF model by selectively sharing user's latent knowledge between domains and preserving the intrinsic geometry of entities within domains.

In this way, more useful knowledge can be transferred to the target domain while avoiding negative transfer.

Findings

Experimental results show that the proposed SKT method achieves best performance compared to seven non-transfer learning and cross-domain recommendation methods.

4. **Log-Based Session Profiling and Online Behavioural Prediction in E–Commerce Websites**

Author: Javier Fabra; Pedro Álvarez; Joaquín Ezpeleta

Year:2020

Doi: https://ieeexplore.ieee.org/document/9200312

Objective

The objective of the research presented in this article advances in this direction by focusing on the behavioural analysis of unregistered customers of an e-commerce, and it shows that it is possible to accurately predict the customer profile of a user session while browsing the website.

Methodology

The proposal requires building prediction models that are used along with clickstream techniques to analyse the customers' behaviours at runtime. To do that, methodologies for server log processing, clustering algorithms and artificial intelligence techniques are combined in a three-phase process. First, the log files are processed to discover the customer profiles. These profiles are then validated and interpreted from a business perspective, associating each one with a set of behavioural patterns that characterize the users belonging to that profile. After

that, a prediction model is created and trained to evaluate users' pattern-based behaviour and determine the customer's profile. Alternately, once the models are available, predictions are conducted using the user's clickstream. This allows the system to perform, after a small number of events, precise predictions concerning the segment the session is probably going to fall into.

Findings

Despite the process being applicable to a wide domain, each step requires specific adaptations when considering its application to specific cases: in the pre-processing phase, where some events are discarded for different reasons that cause events to be considered as non-user events, or non-interesting events, for instance; in the clustering phase, where the vector of features as well as an adequate number of clusters must be established; in the profiling phase, where the ratios between global and cluster values are chosen to define the cluster profiles; in the validation phase, where the cluster-profile relations are evaluated and, perhaps, changed; and in the prediction model synthesis phase, where the number of initial events is chosen as an adequate parameter to obtain an accurate online prediction. Every taken decision is arguable. However, some of the steps are easily automatized. Most clustering tools are able to find an optimal number of clusters, and this is also the case when looking for an adequate number of initial events for prediction, for instance.

5. **A Novel Technique for Behavioural Analytics Using Ensemble Learning Algorithms in E-Commerce.**

   Author: Mohammad Alojail; Surbhi Bhatia

   Year:2020

   Doi: https://ieeexplore.ieee.org/document/9166484

Objective

The main aim is to create a Behavioural Analytics model (say M) based on data fed into an ERP system and use it to drive targeted advertisement. It will create a stream from ERP Database to M Database using technologies like Kafka, Pubnub, Pusher etc. The paper's prime objective is to classify and effectively run targeted advertising using the data that shows user's retail behaviour.

Methodology

This is a novel strategy to take advantage of various ensemble techniques to get better accuracy of the model prediction task; called as ''Ensemble-Blending based Ensemble model''

or ‘‘EBE model’’ in short. All ensemble techniques are application of weak learners on bootstrapped datasets from the original dataset, in EBE model. The second step is to create a multi-class classifier to generate sector-based classification of hot or cold buyer, based on the above example. Say, profile can be marked as – ‘‘hot buyer’’ in cosmetics but ‘‘cold buyer’’ in electronics. Since, this takes a lot of classes into account, for the sake of more accuracy; help of ensemble learning techniques (bagging and boosting algorithms) is the third step to be used for accomplishing the task. A bag of various permutations and combinations of ensemble algorithms will be applied and the one with the highest accuracy will be selected for result as classification.

Findings

The experimental results have shown an accuracy of 76 % with ERP data using the proposed model. Extensive emphasis is put on the selection and analysis of existing ensemble techniques and emphasis was also put on developing various approaches that could be used to give better results, while keeping the underlying concept same. The merit of the model is in its idea where the configurations are flexible.

6. **Aspect-Based Fashion Recommendation with Attention Mechanism**

Author: Weiqian Li; Bugao Xu

Year:2020

Doi: https://ieeexplore.ieee.org/document/9154366

Objective

The objective of this paper is to propose an aspect-based fashion recommendation model with attention mechanism (AFRAM) to predict customer ratings based on online reviews of fashion products.

Methodology

In this paper, the author proposes an aspect-based fashion recommendation model with attention mechanism (AFRAM) based on users’ reviews and ratings to target local and global aspect representations. This model is constructed with two independent paths to process user/item reviews simultaneously, and each path has a convolutional neural network (CNN), a long-short time memory network (LSTM) with attention mechanism to capture local aspect features and global aspect features separately. To enhance the generalization of the AFRAM model, the local and global aspect features from both user and item reviews are merged through mutual operations prior to the rating prediction. Two datasets, the Clothing, Shoes & Jewellery

dataset from Amazon 5-core and the reviews of another national online retailer, are used to train and test AFRAM.

Findings

AFRAM outperformed the five state-of-the-art recommenders in terms of the accuracy of predicting customer ratings on fashion products.

7. **A DNN-Based Cross-Domain Recommender System for Alleviating Cold-Start Problem in E-Commerce**

Author: Hanxin Wang; Daichi Amagata; Takuya Makeawa; Takahiro Hara; Niu Hao; Kei Yonekawa; Mori Kurokawa

Year:2020

Doi: https://ieeexplore.ieee.org/document/9151184

Objective

The objective of the project is to develop a framework specifically designed for alleviating the cold-start problem in e-commerce.

Methodology

The framework employs deep learning to build a cross-domain recommender system based on shared users in these two domains. This is the first attempt that models users based on shared users in online shopping and Ad's domains for solving the user-cold start problem. The author applies Word2Vec to turn textual information on users and items into latent vectors as their representations. Textual information may contain useless information, and Word2Vec cannot capture some structural and semantic correlations between different users.

Findings

The experimental results show the effectiveness of deep neural approaches with knowledge transferred from another domain for the cold-start problem. This framework can recommend products to users of another domain through Ads distribution in a more accurate level.

8. **A Variational Autoencoder Mixture Model for Online Behaviour Recommendation**

Author: Minh-Duc Nguyen; Yoon-Sik Cho

Year:2020

Doi: https://ieeexplore.ieee.org/document/9144583

Objective

The objective of this paper is to develop a mixture model to predict future behaviours of users with a better performance.

Methodology

Variational autoencoder (VAE) is a deep neural network architecture that aims to learn the lower-dimensional representation of data and generate the data from this representation. This model was originally introduced for image processing problems, such as image denoising, but then was applied to other domains such as topic modelling, and also behaviour recommendation. Since VAE is a probabilistic model, it is also convenient to integrate this model into our mixture model.

Findings

Variational autoencoder mixture model (VAE-MM) considers two factors as its main components: (1) the user-specific preference (2) the latent group preference generated through VAE. While the user specific preference reflects the behaviour history of users, the latent group preference represents the hidden interests. The author improved the predictive performance by applying adjustments to the VAE, such as dropout layer or skip connection. VAE-MM considerably outperformed the baseline models. These improvements come from the integration of VAE as a component in mixture model.

9. **A Hybrid Action-Related K-Nearest Neighbour (HAR-KNN) Approach for Recommendation Systems**

Author: Sunkuru Gopal Krishna Patro; Brojo Kishore Mishra; Sanjaya Kumar Panda; Raghvendra Kumar; Hoang Viet Long

Year:2020

Doi: https://ieeexplore.ieee.org/document/9091509

Objective

The objective of the project is to propose recommendation tasks that categorize user's goals to recommend accurate pre-dictions using Hybrid Action-Related K-NN Similarity (HAR-KNN) recommendation system.

Methodology

The author proposes HAR-KNN, an integrated approach of the K-NN algorithm with user behaviour matrix with the following motivations to overcome some problems in existing works: each user rates only a small subset of available products, so most of the cells in the rating matrix are empty. Finding similarity among nearer related users and items is a challenging one. Since the number of items is generated only for a user, hence it is user-specific. It might be difficult to comprehend the taste of the users. This leads to a problem of sparsity. The proposed HAR-KNN method can combine both collaborative and content-based features that categorize user goals and analyse the attributes of many users to form a larger target to accurately predict recommendations. The hybrid filtering solutions help to avoid scalability problems that exist in CB and CF.

Findings

The paper makes proper utilization after constructing the user behaviour matrix utilized to map out the other users with similar tastes and neighbours. It is distinguished by calculating the Euclidean distance and proper improvisation of all the algorithms used in the framework to achieve a similarity level more accurately. Moreover, the HAR-KNN model is validated on the Amazon dataset and effectively compared with different models, CB and CF in terms of MAE, RMSE, MSE, Recall, Precision, and F1-measure values. The experimental result depicts that the performance measures improve high predictive accuracy with low error than traditional methods. Based on present data, the access of user behaviour matrix depends upon the preference of user behaviour.

10. **RACRec: Review Aware Cross-Domain Recommendation for Fully-Cold-Start User**

    Author: Yaru Jin; Shoubin Dong; Yong Cai; Jinlong Hu

    Year:2020

    Doi: https://ieeexplore.ieee.org/document/9042271

Objective

In this paper, a review aware cross-domain recommendation algorithm, called RACRec, is proposed to address the fully cold-start problem in the field of product recommendation. Firstly, reviews are dynamically selected by using the adjacency matrix.

Secondly, domain-specific preference vectors and domain-shared preference vectors of the cold start user are extracted by a migration model.

Methodology

Review-aware cross-domain recommendation algorithm, called RACRec, is proposed. Firstly, a CNN is employed to encode each original review into a vector, which incorporates the sentiment information of the review text. Secondly, valuable reviews are selected for each user and product pair. Thirdly, domain-specific preference vectors in the source domain and target domain and domain-shared preference vectors in the source domain and target domain of the cold start user are extracted by a migration model. On the other hand, the product feature vector in the target domain is generated from review texts by encoder and decoder. The product feature vector is combined with preference vectors of the cold-start user to make the rating prediction.

Findings

Experiments have proven that RACRec works well, and the review selection is indeed indispensable.

11. **Semantics-Aware Autoencoder**

    Author: Vito Bellini; Tommaso Di Noia; Eugenio Di Sciascio; Angelo Schiavone

    Year:2019

    Doi: https://ieeexplore.ieee.org/document/8897546

Objective

The objective of the project is to present SemAuto, a novel approach based on an Autoencoder Neural Network that makes it possible to semantically label neurons in hidden layers, thus paving the way to the model's interpretability and consequently to the explanation of a recommendation

Methodology

The rationale behind the approach is that both Knowledge Graphs and Artificial Neural Networks (ANNs) behind Deep Learning expose a graph-based structure. In a Semantics-Aware Autoencoder (SemAuto), the hidden layers and their connections are substituted by nodes and labelled connections of a KG, thus having an explicit representation of the meaning associated both to hidden nodes and to their mutual connections. This means that each neuron

represents an entity in the adopted KG, and the edge between two autoencoder nodes exist if the corresponding KG entities are connected with a predicate (labelled edge). Inspired by fully-connected Autoencoders, Semantics-Aware ones try to solve the interpretability issue by labelling neurons in hidden layers, thus assigning an attribute in the explicit feature space to each of them. Considering that every hidden neuron represents a feature, it has to be stimulated only if it describes - and thus it belongs to – the associated item.

Findings

Experimental results show that we are able to outperform state of-the-art recommendation algorithms on both accuracy and diversity.

12. **A New Users Rating-Trend Based Collaborative Denoising Auto-Encoder for Top-N Recommender Systems**

Author: Zeshan Aslam Khan; Syed Zubair; Kashif Imran; Rehan Ahmad; Sharjeel Abid Butt; Naveed Ishtiaq Chaudhary

Year:2019

Doi: https://ieeexplore.ieee.org/document/8830343

Objective

In this paper, the author proposes a novel collaborative filtering (CF) strategy to model the user preference trend by giving weights to different rating-trends for top-N recommendations.

Methodology

Proposed a novel users rating-trend based collaborative denoising auto-encoder (UT-CDAE) which determines user-item correlations by evaluating rating-trend (High or Low) of a user towards a set of items. This inclusion of a user's rating-trend provides additional regularization flexibility which helps to predict improved top-N recommendations. According to the proposed UT-CDAE model, the observed user-item interactions in the rating matrix exhibit the rating trend of a user for a set of items. The proposed methodology provides an opportunity to learn the percentage of high trend of a user by calculating the occurrence of high ratings (binary values corresponding to rating values 4 and 5) for items. The users co-liking patterns are also learned well during encoding process when trained on rating-trend based user-oriented nodes. The suggested collaborative filtering strategy (UT-CDAE) proves to be an effective and a unique scheme for recommending specific but related items to the users.

Findings

The proposed method has outperformed other de-noising auto-encoder based methods (DAE and CDAE) for relatively lower values of ICR (0.2 and 0.5). Low ICR values help in retaining a discernible proportion of users' rating-trend in the data set and hence provide valuable information to UT-CDAE to model users' rating behaviour. Comparison of the proposed method with DAE and CDAE in terms of various evaluation metrics show improved performance and robustness of the UT-CDAE for proposing top-N recommendation to the users.

3 SYSTEM ANALYSIS

3.1. EXISTING SYSTEM

The existing e-commerce systems often encounter several limitations and challenges, which hinder their ability to effectively manage inventory and provide personalized recommendations to users.

- **Inventory Management in Traditional Systems**

In a conventional e-commerce system, inventory management often relies on manual or rule-based methods. The system lacks the capability to predict and proactively address stock shortages or excess inventory, leading to challenges in optimizing inventory levels efficiently.

- **Generic Product Recommendations**

Traditional e-commerce platforms offer generic product recommendations that lack personalization. These recommendations are not tailored to individual buyer preferences and behaviors, resulting in a less engaging shopping experience.

EXISTING ALGORITHMS

Existing algorithms used in e-commerce systems encompass a range of techniques aimed at addressing various challenges and optimizing different aspects of the platform. Some common algorithms include:

- **Collaborative Filtering**

Collaborative filtering algorithms analyze user behavior and preferences to generate recommendations. They identify patterns in user interactions, such as purchases, ratings, or browsing history, to recommend items that similar users have liked or interacted with. Collaborative filtering can be user-based, item-based, or model-based, and it's widely used for personalized product recommendations in e-commerce platforms.

- **Content-Based Filtering**

Content-based filtering algorithms recommend items based on their attributes and features, such as product descriptions, categories, or tags. These algorithms analyze the content of items that a user has interacted with or shown interest in and recommend similar items based on their content similarity. Content-based filtering is effective for suggesting products that align with a user's specific preferences or needs.

- **Association Rule Mining**

Association rule mining algorithms identify relationships and patterns between different items in transactional data. They analyze historical purchase data to discover frequent itemsets and association rules, such as "people who bought X also bought Y." These rules are used to generate recommendations for complementary or related products, facilitating cross-selling and up-selling strategies in e-commerce platforms.

- **Clustering Algorithms**

Clustering algorithms group similar items or users together based on their characteristics or behavior. These algorithms identify clusters of items that share common attributes or users who exhibit similar preferences. Clustering techniques such as k-means clustering or hierarchical clustering are used to segment users or products into distinct groups, enabling targeted marketing campaigns and personalized recommendations.

- **Deep Learning Models**

Deep learning algorithms, such as neural networks, convolutional neural networks (CNNs), and recurrent neural networks (RNNs), are increasingly being utilized in e-commerce systems. These models can capture complex patterns and relationships in large volumes of data, enabling more accurate predictions and recommendations. Deep learning techniques are applied in various tasks, including image recognition, natural language processing, and customer behavior analysis, to enhance the capabilities of e-commerce platforms.

These algorithms, along with others not mentioned here, play a crucial role in improving the effectiveness of e-commerce systems by providing personalized recommendations, optimizing inventory management, forecasting demand, and enhancing overall user experiences.

3.1.1. DISADVANTAGES

Drawbacks of Traditional System:

- Manual and time-consuming inventory management processes.
- Lack of personalized recommendations for users.
- Reactive decision-making based on historical data.
- Inefficient operational processes without automation.
- Limited customer engagement due to generic recommendations.
- Challenges in balancing stock levels, leading to stockouts or overstock.

- Privacy concerns with limited transparency and awareness.
- Static product recommendations without real-time updates.
- Manual monitoring and limited automated alerts.
- Security measures may not be as advanced as modern systems.

Drawbacks of Existing Algorithms:

- Cold start problem for new users or items in collaborative filtering.
- Content-based filtering may struggle with diverse user preferences.
- Association rules can be sensitive to outliers and noise.
- Decision trees may overfit to training data, reducing generalization.
- Random Forest may be computationally expensive for large datasets.
- Clustering algorithms can be sensitive to the choice of distance metrics.
- Matrix factorization might face challenges with sparse data.
- Neural networks may require substantial computational resources.
- Bayesian networks may struggle with modeling complex relationships.
- Time series analysis may struggle with non-linear patterns.
- Reinforcement learning may require extensive training for optimal results.

3.2. PROPOSED SYSTEM

The proposed system aims to address the challenges faced by e-commerce platforms by integrating advanced predictive analytics and recommendation models. The system consists of two main components: a predictive analytics component utilizing ARIMA (AutoRegressive Integrated Moving Average) model for stock prediction, and a recommendation engine employing a Decision Tree model for personalized product recommendations.

Predictive Analytics Component: ARIMA Model for Stock Prediction

- The ARIMA model will be utilized to predict and forecast stock levels for sellers.
- Historical sales data, along with external factors such as seasonality and market trends, will be used to train the model.
- The ARIMA model will provide accurate forecasts, enabling sellers to proactively manage inventory levels and avoid stockouts or overstocking situations.
- Alerts will be generated for sellers when potential stock shortages are predicted, allowing them to take preemptive actions to mitigate risks.

Recommendation Engine: Decision Tree Model for Personalized Recommendations:

- The recommendation engine will utilize a Decision Tree model to analyze user preferences and past purchase behavior.
- By considering factors such as browsing history, purchase history, demographic information, and product attributes, the model will generate personalized recommendations for each user.
- The Decision Tree model will dynamically adapt and refine recommendations based on user interactions and feedback, ensuring relevance and accuracy.
- Personalized recommendations will enhance the shopping experience for users, increase engagement, and drive conversions.

Inventory Management Tools for Sellers:

- Provide proactive inventory management tools for sellers, including features for stock monitoring, forecasting, and automated replenishment.
- Implement a user-friendly dashboard interface for sellers to track inventory levels, receive alerts, and make informed decisions.
- Integrate with external systems, such as suppliers or logistics providers, to streamline inventory replenishment processes and minimize stockouts.

Personalized Shopping Experience for Buyers:

- Offer buyers a highly personalized shopping experience by delivering relevant product recommendations tailored to their individual preferences and purchase history.
- Implement user profiling techniques to capture and analyze user preferences, browsing behavior, and demographic information.
- Enhance the user interface with interactive features, such as personalized product carousels and dynamic content recommendations, to engage and retain buyers.

Scalable System Architecture

- Design and implement a scalable system architecture capable of handling large volumes of data and supporting real-time integration with e-commerce platforms.
- Utilize cloud-based infrastructure and microservices architecture to ensure flexibility, scalability, and reliability.
- Employ containerization and orchestration technologies, such as Docker and Kubernetes, to manage and deploy system components efficiently.

3.2.1. ADVANTAGES

- Proactive stock alerts optimize inventory management for sellers.
- Real-time information and streamlined operations improve overall efficiency.
- Personalized next basket recommendations enhance the buyer's shopping experience.
- Enhanced conversion rates lead to increased sales and revenue.
- Continuous monitoring and regular updates ensure adaptability to market changes.
- Privacy-aware algorithms and transparent communication address user privacy concerns.
- Automated monitoring and reporting enable timely issue resolution.
- In-built user feedback mechanisms contribute to continuous improvement.
- Scalable architecture accommodates growing data volumes.

3.3. FEASIBILITY STUDY

Technical Feasibility

- **Assessment of Technology Stack:** The required technologies for predictive analytics (ARIMA model), recommendation engines (Decision Tree model), and scalable infrastructure (cloud-based services) are widely available and well-documented.
- **System Architecture Evaluation:** Preliminary assessments indicate that the proposed micro services architecture and containerization approach can support scalability and real-time integration.
- Availability of Skills and Resources: Skilled personnel proficient in machine learning, data analytics, and cloud computing are available. The necessary resources for development, such as software licenses and development tools, are readily accessible.

Economic Feasibility

- **Cost-Benefit Analysis:** Initial estimates suggest moderate development and implementation costs, primarily driven by software development, infrastructure setup, and training expenses. However, the projected benefits include increased sales revenue, reduced operational costs through optimized inventory management, and enhanced customer satisfaction, potentially resulting in a favorable ROI.
- **Return on Investment (ROI):** Based on conservative projections, the anticipated ROI is expected to exceed the project's costs within the first year of operation, with continued growth and profitability in subsequent years.

Operational Feasibility

- **Integration with Existing Systems:** Compatibility tests indicate that the proposed system can seamlessly integrate with existing e-commerce platforms and databases, with minimal disruption to ongoing operations.
- **User Acceptance Testing:** Stakeholder feedback from user acceptance testing sessions has been overwhelmingly positive, with users expressing enthusiasm for the system's features and functionality.

The project is recommended to proceed to the next phase of development and implementation, with confidence in its potential to deliver significant benefits to stakeholders and drive value for the organization.

4 SYSTEM CONFIGURATION

4.1. HARDWARE SPECIFICATIONS

- **Processor:** Intel Core i5 or equivalent AMD processor
- **RAM:** 8 GB or higher
- **Storage:** 256 GB SSD or higher for faster read/write operations
- **Display:** Full HD (1920 x 1080 pixels) resolution monitor for optimal viewing experience
- **Network Adapter:** Ethernet or Wi-Fi connectivity for internet access

4.2. SOFTWARE SPECIFICATIONS

- **Operating System:** Windows 10 or Ubuntu 20.04 LTS (or later versions) for compatibility with most software packages
- **Development Environment:**
    - Python 3.x: Programming language for application development
    - Flask: Micro web framework for building web applications
    - MySQL: Relational database management system for data storage and retrieval
    - TensorFlow: Open-source machine learning framework for building and training models
    - Pandas: Data manipulation and analysis library for Python
    - Scikit-learn: Machine learning library for Python
    - Matplotlib: Data visualization library for creating plots and charts
    - NumPy: Numerical computing library for mathematical operations
    - Seaborn: Statistical data visualization library based on Matplotlib
- **Web Server:** WampServer (for Windows) or LAMP stack (for Ubuntu) for hosting the web application locally
- **Integrated Development Environment (IDE):** PyCharm, or any preferred Python IDE for code development
- **Browser:** Google Chrome, Mozilla Firefox, or Microsoft Edge for testing and accessing the web application

- **SMTP**

4.3. ABOUT SOFTWARE

4.3.1. PYTHON 3.7.4

Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language. It was created by Guido van Rossum during 1985- 1990. Like Perl, Python source code is also available under the GNU General Public License (GPL). This tutorial gives enough understanding on Python programming language.



Figure 7: Python

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages. Python is a MUST for students and working professionals to become a great Software Engineer specially when they are working in Web Development Domain.

Python is currently the most widely used multi-purpose, high-level programming language. Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java. Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time. Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber… etc. The biggest strength of Python is huge collection of standard library which can be used for the following:

- Machine Learning
- GUI Applications (like Kivy, Tkinter, PyQt etc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like OpenCV, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia

- Scientific computing

Tensor Flow

TensorFlow is an end-to-end open-source platform for machine learning. It has a comprehensive, flexible ecosystem of tools, libraries, and community resources that lets researchers push the state-of-the-art in ML, and gives developers the ability to easily build and deploy ML-powered applications.



Figure 8: TensorFlow

TensorFlow provides a collection of workflows with intuitive, high-level APIs for both beginners and experts to create machine learning models in numerous languages. Developers have the option to deploy models on a number of platforms such as on servers, in the cloud, on mobile and edge devices, in browsers, and on many other JavaScript platforms. This enables developers to go from model building and training to deployment much more easily.

Pandas

pandas are a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the Python programming language. pandas are a Python package that provides fast, flexible, and expressive data structures designed to make working with "relational" or "labeled" data both easy and intuitive. It aims to be the fundamental high-level building block for doing practical, real world data analysis in Python.



Figure 9: Pandas

Pandas is mainly used for data analysis and associated manipulation of tabular data in Data frames. Pandas allows importing data from various file formats such as comma-separated values, JSON, Parquet, SQL database tables or queries, and Microsoft Excel. Pandas allows various data manipulation operations such as merging, reshaping, selecting, as well as data

cleaning, and data wrangling features. The development of pandas introduced into Python many comparable features of working with Data frames that were established in the R programming language. The panda's library is built upon another library NumPy, which is oriented to efficiently working with arrays instead of the features of working on Data frames.

NumPy

NumPy, which stands for Numerical Python, is a library consisting of multidimensional array objects and a collection of routines for processing those arrays. Using NumPy, mathematical and logical operations on arrays can be performed.



Figure 10: NumPy

NumPy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

Matplotlib

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. Matplotlib makes easy things easy and hard things possible.



Figure 11: Matplotlib

Matplotlib is a plotting library for the Python programming language and its numerical mathematics extension NumPy. It provides an object-oriented API for embedding plots into applications using general-purpose GUI toolkits like Tkinter, wxPython, Qt, or GTK.

Scikit Learn

scikit-learn is a Python module for machine learning built on top of SciPy and is distributed under the 3-Clause BSD license.



Figure 12: Scikit-learn

Scikit-learn (formerly scikits.learn and also known as sklearn) is a free software machine learning library for the Python programming language. It features various classification, regression and clustering algorithms including support-vector machines, random forests, gradient boosting, k-means and DBSCAN, and is designed to interoperate with the Python numerical and scientific libraries NumPy and SciPy.

4.3.2. MYSQL

MySQL tutorial provides basic and advanced concepts of MySQL. Our MySQL tutorial is designed for beginners and professionals. MySQL is a relational database management system based on the Structured Query Language, which is the popular language for accessing and managing the records in the database. MySQL is open-source and free software under the GNU license. It is supported by Oracle Company. MySQL database that provides for how to manage database and to manipulate data with the help of various SQL queries. These queries are: insert records, update records, delete records, select records, create tables, drop tables, etc. There are also given MySQL interview questions to help you better understand the MySQL database.



Figure 13: MySQL

MySQL is currently the most popular database management system software used for managing the relational database. It is open-source database software, which is supported by Oracle Company. It is fast, scalable, and easy to use database management system in comparison with Microsoft SQL Server and Oracle Database. It is commonly used in conjunction with PHP scripts for creating powerful and dynamic server-side or web-based enterprise applications. It is developed, marketed, and supported by MySQL AB, a Swedish company, and written in C programming language and C++ programming language. The official pronunciation of MySQL is not the My Sequel; it is My Ess Que Ell. However, you can pronounce it in your way. Many small and big companies use MySQL. MySQL supports many Operating Systems like Windows, Linux, MacOS, etc. with C, C++, and Java languages.

4.3.3. WAMPSERVER

WampServer is a Windows web development environment. It allows you to create web applications with Apache2, PHP and a MySQL database. Alongside, PhpMyAdmin allows you to manage easily your database.



Figure 14: WampServer

WAMPServer is a reliable web development software program that lets you create web apps with MYSQL database and PHP Apache2. With an intuitive interface, the application features numerous functionalities and makes it the preferred choice of developers from around the world. The software is free to use and doesn't require a payment or subscription.

4.3.4. BOOTSTRAP 4

Bootstrap is a free and open-source tool collection for creating responsive websites and web applications. It is the most popular HTML, CSS, and JavaScript framework for developing responsive, mobile-first websites.



Figure 15: Bootstrap 4

It solves many problems which we had once, one of which is the cross-browser compatibility issue. Nowadays, the websites are perfect for all the browsers (IE, Firefox, and Chrome) and for all sizes of screens (Desktop, Tablets, Phablets, and Phones). **Easy to use**: Anybody with just basic knowledge of HTML and CSS can start using Bootstrap

Responsive features: Bootstrap's responsive CSS adjusts to phones, tablets, and desktops

Mobile-first approach: In Bootstrap, mobile-first styles are part of the core framework

Browser compatibility: Bootstrap 4 is compatible with all modern browsers (Chrome, Firefox, Internet Explorer 10+, Edge, Safari, and Opera)

4.3.5. FLASK

Flask is a web framework. This means flask provides you with tools, libraries and technologies that allow you to build a web application. This web application can be some web pages, a blog, a wiki or go as big as a web-based calendar application or a commercial website.

Using an IDE

As good as dedicated program editors can be for your programming productivity, their utility pales into insignificance when compared to Integrated Developing Environments (IDEs), which offer many additional features such as in-editor debugging and program testing, as well as function descriptions and much more.



Figure 16: Flask

Flask is often referred to as a micro framework. It aims to keep the core of an application simple yet extensible. Flask does not have built-in abstraction layer for database handling, nor does it have formed a validation support. Instead, Flask supports the extensions to add such functionality to the application. Although Flask is rather young compared to most Python frameworks, it holds a great promise and has already gained popularity among Python web developers. Let's take a closer look into Flask, so-called "micro" framework for Python. Flask was designed to be easy to use and extend. The idea behind Flask is to build a solid foundation for web applications of different complexity. From then on you are free to plug in any extensions you think you need. Also you are free to build your own modules. Flask is great for all kinds of projects. It's especially good for prototyping. Flask is part of the categories of the micro-framework. Micro-framework is normally framework with little to no dependencies to external libraries. This has pros and cons. Pros would be that the framework

is light, there are little dependency to update and watch for security bugs, cons is that sometime you will have to do more work by yourself or increase yourself the list of dependencies by adding plugins.

4.3.6. SMS API

A SMS API is well-defined software interface which enables code to send short messages via a SMS Gateway. As the infrastructures for SMS communications and the internet are mostly divided, SMS APIs are often used to 'bridge the gap' between telecommunications carrier networks and the wider web. SMS APIs are used to allow web applications to easily send and receive text messages through logic written for standard web frameworks. The SMS API uses HTTP verbs and a RESTful endpoint structure with an access key that is used as the API Authorization. Request and response payloads are formatted as JSON using UTF-8 encoding and URL encoded values.



Figure 17: SMS API

Send & Receive SMS

The SMS API enables users to send single or bulk SMS texts. Highly suited for service related confirmations, updates & reminders. Also for bulk SMS marketing campaigns.

HTTP API: It is used to send single/Multiple SMS from your application. It's typically a simple URL which you should call from your Application to send SMS instantly. We can provide below API's

1. **Send SMS API:** Use this API to send 1 or more sms from your app, response will be a unique **'msgid'**
2. **Delivery Report API:** This API is used to check the delivery status of the sms sent earlier, using **'msgid'** parameter received from step (1).
3. **Balance Check API:** Use this API to check the balance of your account any time.

4. **Receive SMS API:** You can also receive SMS response/feedbacks in real-time using our Virtual Mobile Number or Short Code services

4.3.7. SIMPLE MAIL TRANSFER PROTOCOL

The Simple Mail Transfer Protocol (SMTP) is a technical standard for transmitting electronic mail (email) over a network. Like other networking protocols, SMTP allows computers and servers to exchange data regardless of their underlying hardware or software. Just as the use of a standardized form of addressing an envelope allows the postal service to operate, SMTP standardizes the way email travels from sender to recipient, making widespread email delivery possible.



Figure 18: Simple Mail Transfer Protocol (SMTP)

SMTP is a mail delivery protocol, not a mail retrieval protocol. A postal service delivers mail to a mailbox, but the recipient still has to retrieve the mail from the mailbox. Similarly, SMTP delivers an email to an email provider's mail server, but separate protocols are used to retrieve that email from the mail server so the recipient can read it.

Types of SMTP

RFC 5321 differentiates between four types of SMTP systems:

- **Originating SMTP** is the first system that interacts with the internet as it introduces mail;
- **Delivery SMTP** is the system that receives emails from the internet and delivers them to the recipients;
- **Relay SMTP** relays emails between SMTP servers or MTAs (more on transfer agent meaning below) without modifying the message in any way;
- **Gateway SMTP** or SMTP gateway also transfers emails between different servers but, unlike SMTP relay, it's allowed to transform the message if needed. SMTP gateways are usually firewalls that rewrite addresses or intermediary SMTP servers.

5 SYSTEM DESIGN

5.1. SYSTEM ARCHITECTURE

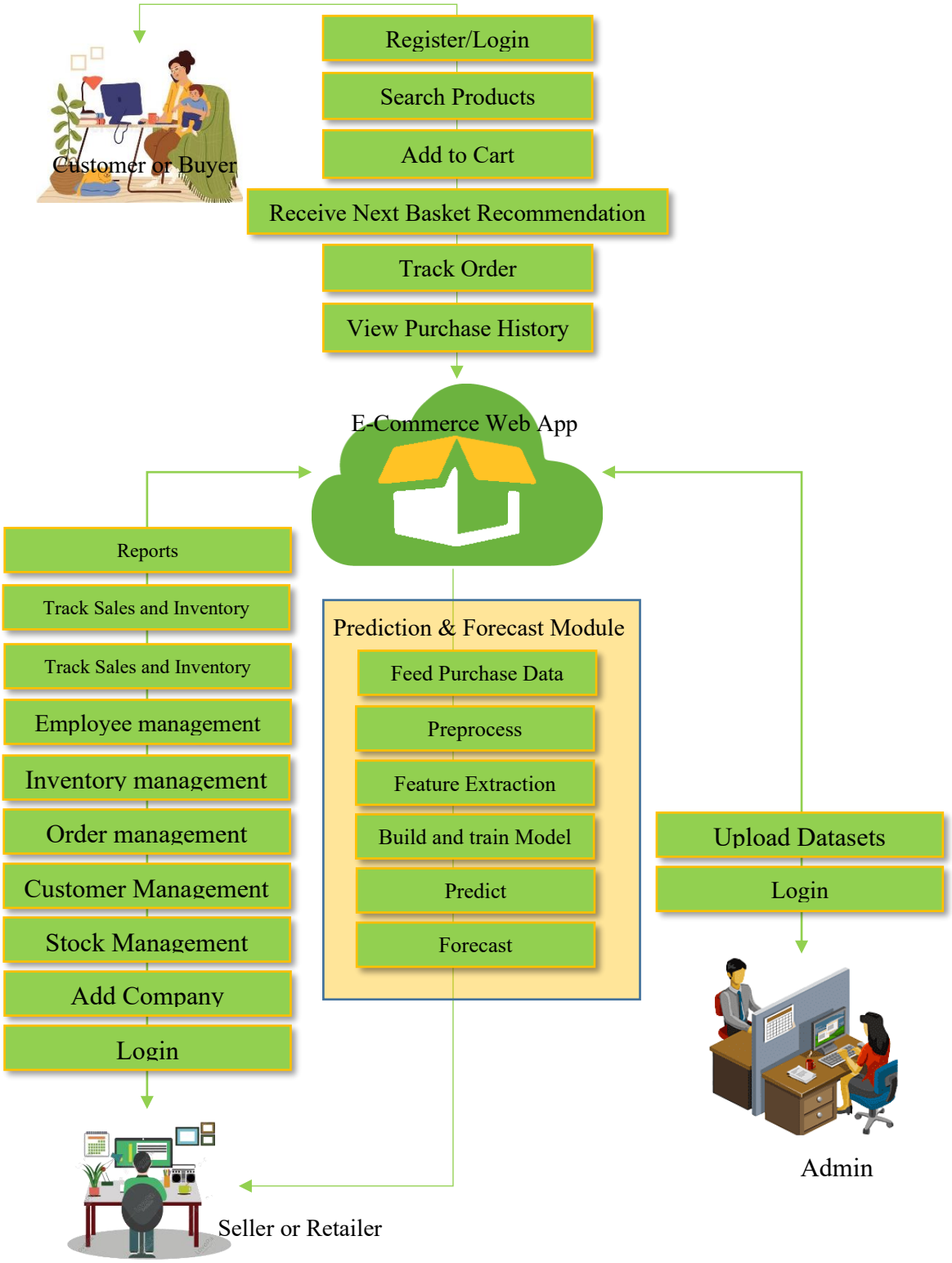


Figure 19: System Architecture

5.2. DATA FLOW DIAGRAM

5.2.1. LEVEL 0

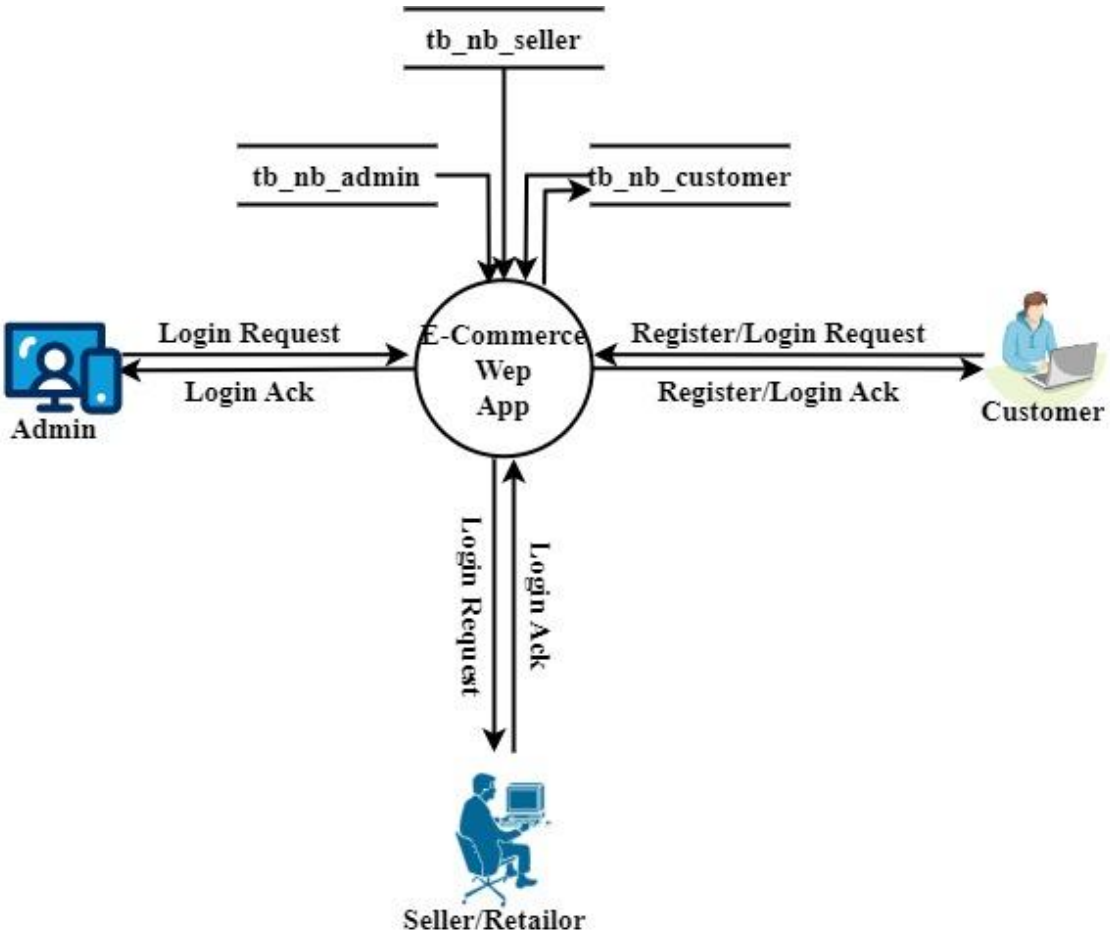


Figure 20: Level 0 Data Flow Diagram

5.2.2. LEVEL 1

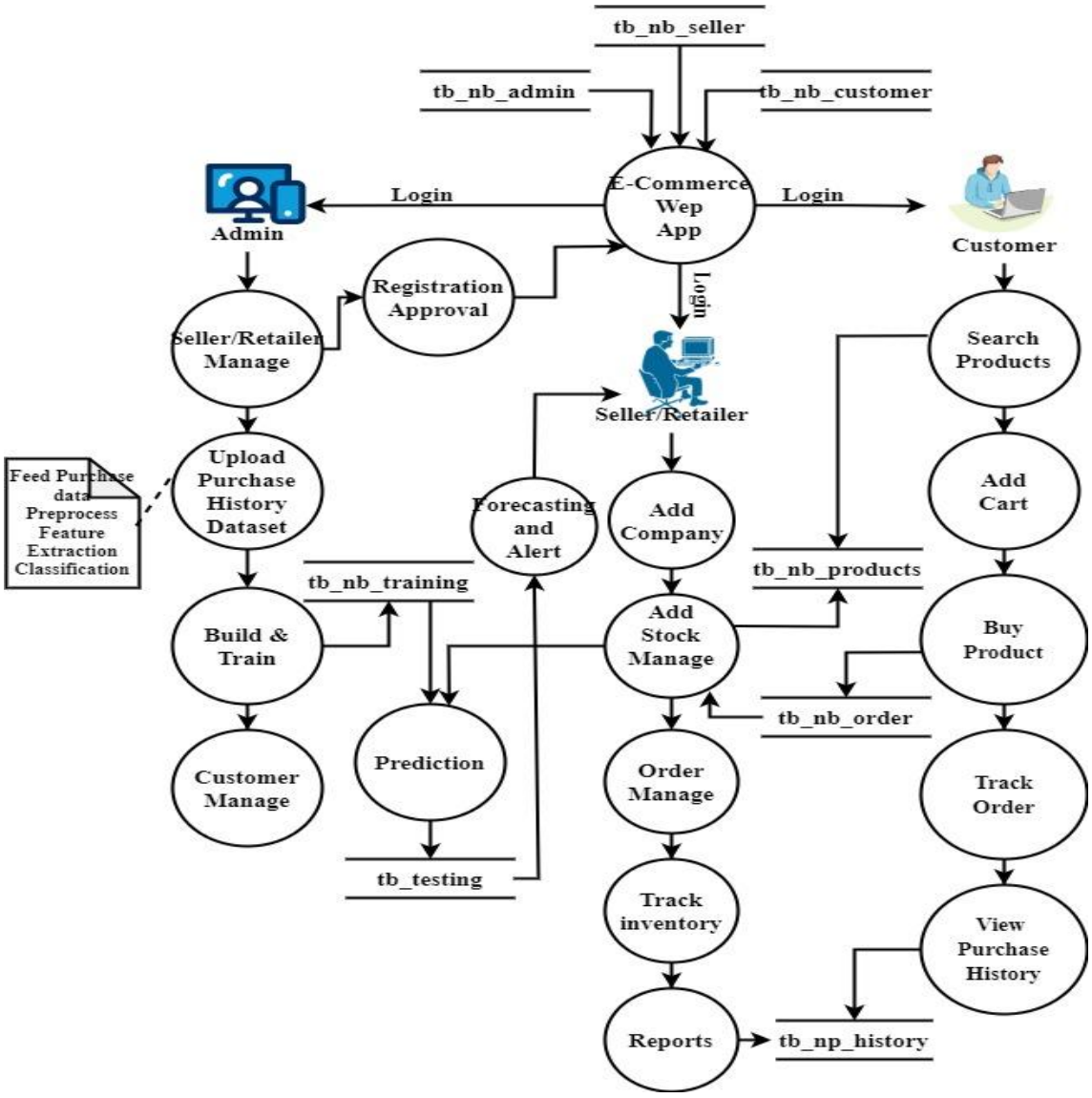


Figure 21: Level 1 Data Flow Diagram

5.2.3. LEVEL 2

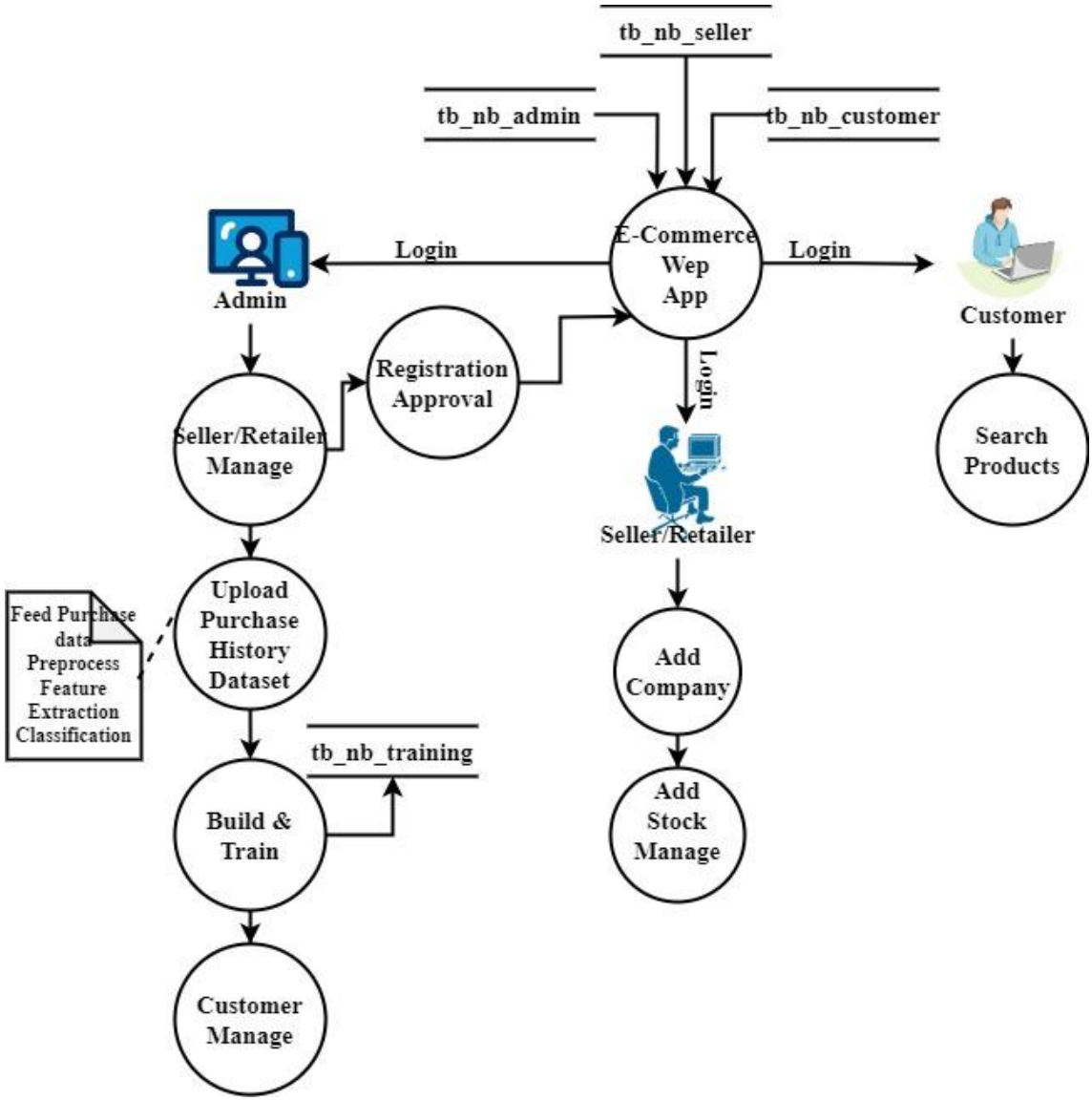


Figure 22: Level 2 Data Flow Diagram

6 METHODOLOGY

6.1. SYSTEM DESCRIPTION

The development of an E-Commerce Web App involves a multifaceted approach integrating various technologies and modules to ensure a seamless online shopping experience. At the forefront of this endeavor is the frontend development, where the user interface is crafted using fundamental web technologies like HTML, CSS, and JavaScript, alongside the Bootstrap framework for responsive design. This aspect focuses on creating an intuitive interface that enables users to effortlessly browse products, manage their shopping carts, and complete purchases. On the backend, Python, with the Flask web framework, takes center stage to handle server-side operations and business logic. Flask provides a lightweight yet powerful framework for rapid development, facilitating tasks such as user authentication, product management, and order processing. The backend also encompasses the creation of API endpoints that enable communication between the frontend interface and the server, allowing for seamless interaction and data exchange. Integral to the functionality of the E-Commerce Web App is the database management system, MySQL, which stores and manages various types of data critical to the platform's operation. The design of the database schema is meticulously crafted to ensure data integrity and efficient query execution, with careful consideration given to relationships between different entities such as user information, product details, and order records. This robust database infrastructure enables seamless data retrieval, storage, and manipulation, ensuring the smooth functioning of the application. Furthermore, the integration of advanced technologies such as machine learning plays a pivotal role in enhancing the platform's capabilities. Machine learning models, leveraging libraries like TensorFlow, Pandas, and Scikit-learn, are integrated into the backend to enable functionalities such as sales forecasting and personalized product recommendations. By analyzing historical data and user behavior patterns, these models provide valuable insights that optimize inventory management, anticipate demand fluctuations, and enhance the overall shopping experience for users. Additionally, the E-Commerce Web App incorporates specialized modules such as the Forecaster Model and Recommendation Engine, which further enhance its functionality and user engagement. The Forecaster Model and Recommendation Engine are integral components of the E-Commerce Web App, enhancing its capabilities and providing valuable insights to both users and administrators. The Forecaster Model begins with the collection of historical purchase data from various sources, including databases and external APIs. This data,

encompassing timestamps, product IDs, and quantities sold, is then imported into the analysis environment for further processing. Through data visualization techniques such as time series plots and histograms, patterns and anomalies within the data are identified, paving the way for preprocessing steps. Null values, misspelled entries, and redundant data points are addressed during preprocessing to ensure data quality. Feature extraction plays a crucial role in uncovering relevant sales patterns, achieved through the construction of a co-occurrence matrix capturing product relationships. Subsequently, the Forecaster Model is built and trained using the ARIMA technique, leveraging preprocessed and feature-enriched data. Once trained, the ARIMA model is deployed within the E-Commerce Web App, providing real-time sales forecasts that inform inventory management decisions and strategic planning. On the other hand, the Recommendation Engine begins with the collection of diverse datasets, including browsing behavior, purchase history, and demographic information, to understand user preferences comprehensively. Following data import and visualization, preprocessing ensures dataset cleanliness, while feature selection and extraction identify key features influencing recommendations. The recommendation engine is then built and trained using decision tree modeling, leveraging preprocessed data to generate personalized product recommendations. Once trained, the decision tree model is deployed within the E-Commerce platform, seamlessly integrating into the backend infrastructure. This allows for real-time recommendation generation based on user interactions, ensuring continuous adaptation to evolving user preferences. Together, the Forecaster Model and Recommendation Engine enhance user experience, drive sales, and support informed decision-making within the E-Commerce ecosystem. The Sales Forecasting module within the E-Commerce platform plays a pivotal role in anticipating future sales trends, thereby enabling proactive responses to market dynamics and enhancing operational efficiency. This module is responsible for meticulously tracking and recording customer purchases within the E-Commerce platform. It captures crucial information such as the products purchased, quantities, timestamps, and unique customer identifiers. By logging each purchase transaction, the system maintains a comprehensive record of sales data, which serves as the foundation for subsequent forecasting activities.

The primary objective of this module is to predict future sales trends by leveraging historical sales data and other relevant factors. Utilizing advanced forecasting techniques such as the ARIMA model, time series analysis, or machine learning algorithms, the module generates forecasts for future sales volumes or revenues. Through the analysis of historical patterns and trends, the module anticipates potential demand fluctuations, seasonal variations,

and emerging market trends, empowering stakeholders with actionable insights. Once sales forecasts are generated, this module takes charge of notifying stakeholders about potential changes in demand or sales projections. Alerts are triggered based on predefined thresholds or significant deviations from expected sales patterns. Stakeholders, including inventory managers, sales teams, and executives, receive real-time notifications and alerts via email, SMS, or within the E-Commerce platform's dashboard. These alerts prompt timely action, such as adjusting inventory levels, replenishing stock, or revising marketing strategies, aligning with forecasted demand to optimize sales performance. Through seamless integration of these components, the Sales Forecasting module equips the E-Commerce platform with the capability to anticipate market dynamics, optimize inventory management, and maximize revenue opportunities, ultimately enhancing the overall efficiency and profitability of the platform.

SYSTEM FLOW

1. **User Interaction:** Users access the E-Commerce platform via web browsers or mobile applications.
2. **Registration/Login:** New users register for an account or existing users log in using their credentials to access the platform.
3. **Browsing Products:** Users browse through the available products, categories, and promotions offered on the platform.
4. **Adding to Cart:** Users select products they wish to purchase and add them to their shopping carts.
5. **Checkout Process:** Users proceed to the checkout process to review their selected items, input delivery details, and choose payment options.
6. **Order Placement:** Users confirm their orders, initiating the purchase process.
7. **Order Processing:** Upon order confirmation, the system processes the order, verifies payment, and sends notifications to the user and administrators.
8. **Inventory Management:** Administrators manage inventory levels based on incoming orders, stock availability, and forecasts generated by the Forecaster Model.
9. **Seller Interaction:** Sellers receive notifications of new orders and update product availability and order statuses accordingly.
10. **Forecasting:** The Forecaster Model analyzes historical purchase data to predict future sales trends and demand fluctuations.

11. **Recommendation Generation:** The Recommendation Engine generates personalized product recommendations based on user behavior and preferences.
12. **Notification Delivery:** Stakeholders, including users, administrators, and sellers, receive notifications via SMS, email, or in-app messages regarding order statuses, inventory updates, and recommended products.
13. **Feedback Collection:** Users have the option to provide feedback on their shopping experience, products purchased, and overall satisfaction with the platform.

6.2. MODULES DESCRIPTION

6.2.1. E-Commerce Web App

The design and development of an E-Commerce Web App using Python, Flask, MySQL, Bootstrap, and Wampserver, along with TensorFlow, Pandas, Scikit-learn, Matplotlib, NumPy, Seaborn, and NLP packages involves several steps. The E-Commerce Web App integrates several essential modules to ensure a smooth and efficient online shopping experience. With the Product Management Module, administrators can effortlessly manage the product catalog, while users enjoy easy browsing and selection. Seamless user authentication and account management features guarantee secure access and personalized interactions. The Shopping Cart Module facilitates hassle-free checkout, including transparent cost calculation and multiple payment options for user convenience. Subsequently, the Order Management Module ensures timely order processing and delivery tracking, while the Customer Support Module offers responsive assistance through FAQs, live chat, and ticketing systems. Administrators benefit from insightful analytics provided by the Reporting and Analytics Module, guiding strategic decisions, while the Content Management Module empowers them to maintain a dynamic and engaging online presence. Together, these modules create a robust E-Commerce Web App that enhances user satisfaction and streamlines administrative operations.

Frontend Development:

The frontend of the E-Commerce Web App is responsible for creating an engaging and user-friendly interface that allows users to browse products, add items to their cart, and complete purchases. It comprises several key elements:

- **HTML, CSS, and JavaScript:** These fundamental web technologies are used to structure web pages, style elements, and add interactivity, respectively.

- **Bootstrap Framework:** Leveraging Bootstrap simplifies frontend development by providing pre-designed UI components and responsive layout grids.
- **User Interface Components:** Navigation menus, product listings, shopping carts, and checkout forms are meticulously crafted to ensure intuitive navigation and seamless user interactions.
- **Client-Side Interactivity:** JavaScript is employed to implement dynamic features such as product filtering, form validation, and interactive elements, enhancing the overall user experience.

Backend Development:

The backend serves as the backbone of the E-Commerce Web App, handling server-side operations and business logic. It encompasses various crucial aspects:

- **Python Language:** Backend logic is implemented using Python, a versatile and widely used programming language.
- **Flask Web Framework:** Flask is employed as a lightweight web framework to facilitate rapid and efficient backend development.
- **API Endpoints:** The backend comprises API endpoints that manage user authentication, product management, order processing, and other essential functionalities.
- **User Authentication:** Secure user authentication mechanisms are implemented to ensure the confidentiality and integrity of user accounts.
- **Data Processing:** Server-side data processing operations include generating product recommendations, handling payments, and managing inventory.
- **Integration with Machine Learning Models:** Advanced functionalities such as sales forecasting and personalized recommendations are achieved by integrating machine learning models into the backend.

Database Management:

The database is a crucial component of the E-Commerce Web App, responsible for storing and managing various types of data, including user information, product details, and order records. Key aspects of database management include:

- **MySQL Database:** MySQL is utilized as the relational database management system (RDBMS) for storing structured data efficiently.

- **Database Schema Design:** The structure of the database tables, along with relationships between entities, is meticulously designed to ensure data integrity and efficient query execution.
- **Data Storage:** User profiles, product catalogs, order history, and other application data are stored securely within the database.
- **Data Retrieval and Manipulation:** The backend server interacts with the database to retrieve, store, and manipulate data based on user actions and system requirements.
- **Integration with Backend:** Seamless connectivity between the backend server and the database ensures efficient data retrieval, storage, and manipulation, enabling the smooth functioning of the E-Commerce Web App.

Together, these components form a cohesive system that powers the E-Commerce Web App, providing users with a seamless shopping experience while ensuring efficient data management and processing on the server-side.

6.2.2. Forecaster Model

6.2.2.1. Data Collection

In the initial stage of the Forecaster Model, historical purchase data is collected from various sources such as databases or external APIs. This module employs database queries or API calls to extract relevant information, including timestamps indicating when purchases were made, quantities of products sold, and corresponding product IDs. By compiling this historical data, the model gains insight into past purchasing patterns, essential for forecasting future sales.

6.2.2.2. Import Dataset

Once the historical purchase data is collected, it is imported into the analysis environment. This step involves loading the dataset into a suitable data structure, such as a pandas DataFrame in Python, to facilitate further processing and analysis. Importing the dataset effectively sets the stage for subsequent data exploration and manipulation.

6.2.2.3. Data Visualization

Before proceeding with data pre-processing, it is beneficial to visualize the dataset to gain insights and identify potential patterns or anomalies. Data visualization techniques such as time series plots, histograms, and scatter plots are employed to understand the distribution and trends present in the data. Visualization aids in identifying seasonality, trends, and outliers, which can inform subsequent data processing steps.

6.2.2.4. Pre-processing

Once the data is collected, it undergoes preprocessing to ensure its quality and suitability for analysis. This module addresses common data issues such as null values, misspelled entries, and redundant data points. Null values are handled by imputation or removal, ensuring that missing information does not compromise the accuracy of the model. Additionally, misspelled data is corrected, and redundant entries are eliminated to streamline further analysis. Through this preprocessing step, the data is cleansed and standardized, ready for feature extraction and model training.

6.2.2.5. Feature Extraction

Feature extraction involves identifying and extracting relevant features from the pre-processed data that contribute to sales patterns. In this module, a co-occurrence matrix is constructed to capture relationships between products based on their frequency of occurrence together in transactions. This matrix helps uncover significant associations and dependencies among products, which can influence future sales. By extracting meaningful features, the model gains insight into the underlying dynamics of the sales data, improving its predictive capabilities.

6.2.2.6. Forecaster Model Build and Train using ARIMA Model

At the core of the Forecaster Model lies the construction and training of the ARIMA (AutoRegressive Integrated Moving Average) model. ARIMA is a powerful time series forecasting technique capable of capturing complex patterns and trends in sequential data. In this module, the pre-processed and feature-enriched data is used to train the ARIMA model. The model parameters are fine-tuned through iterative processes to optimize predictive accuracy. By leveraging historical purchase data, the ARIMA model learns patterns and seasonality in sales, enabling it to generate forecasts for future time periods.

6.2.2.7. Deploy the Model

Once the ARIMA model is trained and validated, it is deployed within the E-Commerce Web App for real-time forecasting. This module involves integrating the trained model into the backend infrastructure of the application, allowing it to make predictions based on incoming data. The deployed model continuously monitors and analyzes new sales data, updating its forecasts dynamically to reflect changing market conditions. Through seamless integration, the Forecaster Model provides valuable insights to stakeholders, informing inventory management decisions and supporting strategic planning within the E-Commerce platform.

6.2.3. Recommendation Engine (Decision Tree): Build and Train

6.2.3.1. Data Collection

Data collection is a fundamental step in building a recommendation engine. It involves gathering diverse datasets that provide insights into user behavior, preferences, and interactions with the E-Commerce platform. These datasets can include browsing behavior, purchase history, product reviews, demographic information, and more. Both explicit (directly provided by the user) and implicit (inferred from user actions) data collection methods are utilized to capture a comprehensive view of user preferences and characteristics.

Data Sources:

- **Browsing Behavior:** Pages visited, time spent on each page, clicks, and interactions.
- **Purchase History:** Previous purchases, order details, and transaction history.
- **Demographic Information:** Gender, age, location, marital status, etc.
- **Behavioral Data:** Actions such as add-to-cart, wishlist, searches, etc.
- **Affinity-Based Data:** User interests, preferences, and affinity towards specific products or categories.
- **Online and Offline Purchase Data:** Products purchased online or in-store, including frequency and preferences.

6.2.3.2. Import Dataset

Once the relevant datasets are collected, they are imported into the analysis environment. This step involves loading the datasets into a suitable data structure (e.g., pandas DataFrame) for further processing and analysis.

6.2.3.3. Data Visualization

Data visualization plays a crucial role in understanding the characteristics and patterns within the datasets. Visualizations such as histograms, scatter plots, and heatmaps are utilized to explore the distribution of data, identify trends, and detect outliers. This step helps in gaining insights into the underlying patterns present in the data, which can inform subsequent processing steps.

6.2.3.4. Pre-processing

Pre-processing ensures that the dataset is clean and suitable for analysis. This involves handling null values, correcting misspelled data, and removing redundant rows. Techniques

such as imputation, data validation, and data deduplication are applied to ensure data integrity and accuracy.

6.2.3.5. Feature Selection

Feature selection is crucial for building an effective recommendation engine. This module selects relevant features from the dataset that contribute most to the recommendation process. Techniques such as matrix factorization are employed to identify latent features and reduce dimensionality while retaining meaningful information.

6.2.3.6. Feature Extraction

Feature extraction aims to capture relationships between different features in the dataset. A correlation matrix is constructed to quantify the strength and direction of correlations between variables. This helps identify which features are most strongly correlated with user preferences and can influence recommendations.

6.2.3.7. Recommender Engine Build and Train

The core of the recommendation engine involves building and training a decision tree model. Decision trees are a popular choice for recommendation systems due to their interpretability and ability to handle both categorical and numerical data. The decision tree is trained on the pre-processed and feature-selected dataset, learning patterns and relationships between user attributes and preferences.

6.2.3.8. Deploy the Model

Once the decision tree model is trained and validated, it is deployed within the E-Commerce platform for real-time recommendation generation. The model is integrated into the platform's backend infrastructure, allowing it to generate personalized recommendations based on user interactions and preferences. Continuous monitoring and updates ensure that the recommendation engine remains effective and adapts to changing user behavior and preferences.

6.2.4. Sales Forecasting

E-Commerce platform can effectively forecast future sales trends, anticipate demand fluctuations, and proactively respond to market dynamics, ultimately enhancing operational efficiency and maximizing revenue opportunities.

6.2.4.1. Customer Buy Product

This module tracks and records customer purchases within the E-Commerce platform. It captures information such as the products purchased, quantities, timestamps, and customer identifiers. Each purchase transaction is logged in the system to maintain a comprehensive record of sales data.

6.2.4.2. Predict Sales Forecasting

The primary objective of this module is to predict future sales trends based on historical sales data and other relevant factors. Utilizing advanced forecasting techniques such as the ARIMA model (AutoRegressive Integrated Moving Average), time series analysis, or machine learning algorithms, this module generates forecasts for future sales volumes or revenues. By analyzing historical patterns and trends, the module predicts potential demand fluctuations, seasonal variations, and emerging market trends.

6.2.4.3. Notify and Alert

Once sales forecasts are generated, this module is responsible for notifying stakeholders about potential changes in demand or sales projections. Alerts may be triggered based on predefined thresholds or significant deviations from expected sales patterns. Stakeholders, including inventory managers, sales teams, and executives, receive real-time notifications and alerts via email, SMS, or within the E-Commerce platform's dashboard. These alerts prompt timely action, such as adjusting inventory levels, replenishing stock, or revising marketing strategies, to align with forecasted demand and optimize sales performance.

6.2.5. Next Basket Recommendation

E-Commerce platform can enhance the shopping experience for customers by providing personalized and relevant product recommendations, ultimately driving customer satisfaction, loyalty, and revenue growth.

6.2.5.1. Customer Add Product to Cart

This module tracks customer interactions within the E-Commerce platform, specifically when a customer adds products to their shopping cart. It captures information such as the products added, quantities, timestamps, and customer identifiers. Each cart addition event is logged in the system to facilitate personalized recommendation generation.

6.2.5.2. Predict Product Recommendation

Using advanced recommendation algorithms, this module predicts the next set of products that a customer is likely to purchase based on their current shopping cart contents and browsing history. Techniques such as collaborative filtering, content-based filtering, or hybrid approaches may be employed to generate personalized product recommendations. The algorithm analyzes customer preferences, purchase history, product attributes, and other relevant factors to suggest complementary or related items that align with the customer's interests and preferences.

6.2.5.3. Notify Next Basket

Once the next basket of recommended products is generated, this module notifies the customer about the suggested items. Notifications may be delivered in real-time via email, push notifications, or displayed within the E-Commerce platform's user interface. The goal is to engage customers and encourage them to explore and potentially purchase the recommended products. Additionally, the platform may provide incentives such as discounts or promotions to incentivize customers to complete their purchase and enhance overall sales performance.

6.2.6. Actors Dashboard

6.2.6.1. E-Commerce Admin

- **Login:** E-Commerce Admin can securely log in to the admin dashboard using unique credentials.
- **Approve Seller:** Admin can review and approve seller registrations, ensuring compliance with platform policies and standards.
- **Build Model:** Admin has access to tools and functionalities to build and train predictive models for sales forecasting and recommendation engines.
- **User Management:** Admin can manage user accounts, including creating, updating, and deactivating accounts as needed.

6.2.6.2. Customer or Buyer

- **Register:** Customers can create a new account by providing necessary details such as name, email, and password.
- **Login:** Registered customers can securely log in to their account using credentials.
- **Search Product:** Customers can browse and search for products using keywords, categories, or filters.

- **Add to Cart:** Customers can add selected products to their shopping cart for future purchase.
- **Receive Next Basket Recommendation:** Customers receive personalized recommendations for the next basket of products based on their shopping cart contents and browsing history.
- **Add Recommended Products to Cart:** Customers can review and add recommended products to their shopping cart.
- **Place Order:** Customers can proceed to checkout and place orders for selected products.
- **Pay Order:** Customers can make payments securely using various payment options integrated into the platform.
- **Track Order:** Customers can track the status of their orders in real-time, from processing to delivery.
- **Receive Order:** Customers receive notifications and updates when their orders are dispatched and delivered.
- **View Transaction History:** Customers can view their transaction history, including past orders and payments.
- **Provide Feedback:** Customers can leave reviews and feedback for products and their overall shopping experience.

6.2.6.3. Seller

- **Registration:** Sellers can register on the platform by providing necessary information and agreeing to terms and conditions.
- **Receive Registration Approval:** Sellers receive notifications once their registration is approved by the admin.
- **Login:** Registered sellers can securely log in to their account using credentials.
- **Update Seller Profile:** Sellers can update their profile information, including contact details and product listings.
- **Stock Management:** Sellers can manage their product inventory, including adding new products, updating stock levels, and removing discontinued items.
- **Customer Management:** Sellers can manage customer interactions, including responding to inquiries, resolving issues, and processing returns or refunds.
- **Order Management:** Sellers can view and manage incoming orders, including order processing, packaging, and shipping.

- **Receive Next Month Forecast:** Sellers receive forecasts for future sales and demand, enabling proactive inventory planning and management.
- **Receive Stock Alert:** Sellers receive alerts and notifications for low stock levels or potential stockouts, prompting timely replenishment actions.
- **Inventory Management:** Sellers can track and manage their inventory levels, ensuring adequate stock availability to meet customer demand.
- **Track Sales and Inventory:** Sellers can monitor sales performance and inventory metrics, including sales volume, revenue, and stock turnover.
- **View Reviews:** Sellers can view customer reviews and ratings for their products, gaining insights into customer feedback and satisfaction.
- **View Transaction History:** Sellers can access transaction history, including sales orders, payments, and customer interactions.

6.2.7. Notification

The notification modules in the E-Commerce platform utilize various channels such as SMS, email, and in-app notifications to keep stakeholders informed and engaged. Customers receive updates on their orders, promotions, abandoned carts, and product restocks through SMS, email, and in-app messages, ensuring they stay informed regardless of their preferred communication channel. Administrators are alerted via SMS, email, and in-app notifications about new orders, low inventory levels, sales reports, and user activities, enabling them to manage operations effectively. Sellers receive notifications through SMS, email, and in-app messages regarding new orders, product feedback, and inventory updates, facilitating timely actions and efficient inventory management. These multi-channel notification systems ensure seamless communication, proactive decision-making, and enhanced user satisfaction throughout the platform.

6.3. SYSTEM IMPLEMENTATION

The system implementation of the project involves several key steps to transform the design concepts into a functional and operational E-Commerce platform. Here's an overview of the implementation process:

1. **Environment Setup:**
    - Set up development environments for frontend and backend development.
    - Install necessary software tools, frameworks, and libraries, including IDEs, version control systems, and dependencies.

2. **Database Design and Setup:**
    - Design the database schema to efficiently store and manage data related to users, products, orders, and transactions.
    - Set up the database management system (e.g., MySQL) and create tables, relationships, and constraints based on the design.
3. **Frontend Development:**
    - Develop the user interface using HTML, CSS, and JavaScript to create visually appealing and interactive web pages.
    - Utilize frontend frameworks like Bootstrap to ensure responsiveness and compatibility across devices.
4. **Backend Development:**
    - Implement server-side logic using Python and the Flask web framework to handle client requests and perform business operations.
    - Develop RESTful API endpoints for functionalities such as user authentication, product management, order processing, and data retrieval.
5. **Integration of Machine Learning Models:**
    - Integrate machine learning models (e.g., ARIMA for sales forecasting, decision tree for recommendation) into the backend infrastructure.
    - Train and fine-tune the models using historical data to enhance their accuracy and effectiveness.
6. **Data Integration:**
    - Connect the backend server to the database system to enable data storage, retrieval, and manipulation.
    - Implement data validation and sanitization mechanisms to ensure data integrity and security.
7. **Notification Module Implementation:**
    - Develop modules to handle notifications via SMS, email, and in-app messages.
    - Integrate third-party services (e.g., Twilio for SMS, SMTP for email) to facilitate notification delivery.
8. **Testing and Quality Assurance:**
    - Conduct comprehensive testing of the entire system to identify and rectify bugs, errors, and inconsistencies.
    - Perform unit tests, integration tests, and system tests to ensure functionality, reliability, and performance.

9. **Deployment:**
    - Deploy the E-Commerce platform to a production environment, such as cloud hosting services or dedicated servers.
    - Configure server settings, security measures, and scalability options to meet operational requirements.
10. **Monitoring and Maintenance:**
    - Implement monitoring tools to track system performance, usage metrics, and error logs.
    - Regularly update and maintain the software to address security vulnerabilities, performance issues, and user feedback.
11. **User Training and Support:**
    - Provide training and support resources for administrators, sellers, and customers to effectively use the platform.
    - Offer documentation, tutorials, and customer service channels for assistance and troubleshooting.

By following these steps, the system implementation ensures the successful development and deployment of the E-Commerce platform, enabling seamless online shopping experiences and leveraging machine learning capabilities for enhanced functionality and user satisfaction.

6.4. SYSTEM TESTING

System testing is a crucial phase in the development process of the E-Commerce platform, ensuring that the system functions correctly, meets requirements, and delivers a seamless user experience. Here's an overview of the system testing process:

1. **Unit Testing:**
    - Test individual components, modules, or functions of the system in isolation.
    - Verify that each unit performs its intended function accurately.
    - Use testing frameworks and tools to automate unit tests and ensure code quality.
2. **Integration Testing:**
    - Test the interaction between different components or modules of the system.
    - Verify that integrated units function correctly together and exchange data as expected.

- Identify and resolve any integration issues, such as communication errors or compatibility issues.

3. **Functional Testing:**
    - Test the functionality of the entire system against specified requirements.
    - Conduct tests for user authentication, product browsing, adding items to the cart, checkout process, order management, and other core functionalities.
    - Ensure that all features work as intended and deliver the desired outcomes.
4. **User Interface Testing:**
    - Evaluate the user interface for consistency, usability, and responsiveness.
    - Test navigation flows, form validations, error handling, and visual elements across different devices and screen sizes.
    - Ensure that the interface adheres to design principles and provides an intuitive user experience.
5. **Performance Testing:**
    - Assess the performance of the system under various load conditions.
    - Conduct tests to measure response times, throughput, and resource utilization.
    - Identify bottlenecks, scalability issues, or performance degradation under high traffic scenarios.
6. **Security Testing:**
    - Evaluate the security measures implemented within the system.
    - Conduct vulnerability assessments, penetration testing, and authentication checks to identify potential security risks.
    - Verify compliance with security standards and protocols to protect user data and prevent unauthorized access.
7. **Compatibility Testing:**
    - Test the compatibility of the system with different web browsers, operating systems, and devices.
    - Ensure consistent functionality and display across platforms to provide a seamless user experience for all users.
8. **Regression Testing:**
    - Conduct regression tests to verify that recent code changes or updates have not introduced new defects or regressions.
    - Re-run previously executed tests to ensure that existing functionalities remain unaffected by changes.

9. **User Acceptance Testing (UAT):**
    - Involve stakeholders, including administrators, sellers, and customers, in user acceptance testing.
    - Gather feedback and validate that the system meets business requirements and user expectations.
    - Address any identified issues or concerns before proceeding to production deployment.

6.5. TEST CASES

Test Case ID: TC001

Input: E-Commerce Admin attempts to log in with valid credentials.

Expected Result: Admin securely logs in to the admin dashboard.

Actual Result: Admin successfully logged in.

Status: Pass

Test Case ID: TC002

Input: Admin reviews and approves a seller registration.

Expected Result: Seller registration is approved, ensuring compliance with platform policies.

Actual Result: Seller registration successfully approved.

Status: Pass

Test Case ID: TC003

Input: Admin accesses tools to build and train predictive models.

Expected Result: Admin can access tools and functionalities for building and training predictive models.

Actual Result: Admin accessed tools successfully.

Status: Pass

Test Case ID: TC004

Input: Admin manages user accounts (create, update, deactivate).

Expected Result: Admin can create, update, and deactivate user accounts as needed.

Actual Result: User accounts managed successfully.

Status: Pass

Test Case ID: TC005

Input: Customer registers a new account.

Expected Result: Customer successfully creates a new account by providing necessary details.

Actual Result: Customer registered successfully.

Status: Pass

Test Case ID: TC006

Input: Registered customer logs in with valid credentials.

Expected Result: Customer securely logs in to their account.

Actual Result: Customer logged in successfully.

Status: Pass

Test Case ID: TC007

Input: Customer searches for a product using keywords.

Expected Result: Customer can browse and search for products using keywords, categories, or filters.

Actual Result: Product search successful.

Status: Pass

Test Case ID: TC008

Input: Customer adds a product to the shopping cart.

Expected Result: Customer can add selected products to their shopping cart.

Actual Result: Product added to the shopping cart successfully.

Status: Pass

Test Case ID: TC009

Input: Customer receives personalized recommendations for the next basket of products.

Expected Result: Customer receives relevant product recommendations based on their shopping cart contents and browsing history.

Actual Result: Recommendations provided successfully.

Status: Pass

Test Case ID: TC010

Input: Customer proceeds to checkout and places an order for selected products.

Expected Result: Customer can place orders securely for selected products.

Actual Result: Order placed successfully.

Status: Pass

Test Case ID: TC011

Input: Seller registers on the platform by providing necessary information.

Expected Result: Seller successfully registers on the platform.

Actual Result: Seller registered successfully.

Status: Pass

Test Case ID: TC012

Input: Seller logs in with valid credentials.

Expected Result: Seller securely logs in to their account.

Actual Result: Seller logged in successfully.

Status: Pass

Test Case ID: TC013

Input: Seller updates their profile information, including contact details and product listings.

Expected Result: Seller can update their profile information.

Actual Result: Profile information updated successfully.

Status: Pass

Test Case ID: TC014

Input: Seller manages their product inventory, including adding new products, updating stock levels, and removing discontinued items.

Expected Result: Seller can effectively manage their product inventory.

Actual Result: Inventory managed successfully.

Status: Pass

Test Case ID: TC015

Input: Seller responds to inquiries, resolves issues, and processes returns or refunds.

Expected Result: Seller effectively manages customer interactions.

Actual Result: Customer interactions managed successfully.

Status: Pass

Test Case ID: TC016

Input: Seller views and manages incoming orders, including order processing, packaging, and shipping.

Expected Result: Seller can efficiently manage incoming orders.

Actual Result: Orders managed successfully.

Status: Pass

Test Case ID: TC017

Input: Seller receives forecasts for future sales and demand.

Expected Result: Seller receives accurate forecasts for future sales.

Actual Result: Forecasts received successfully.

Status: Pass

Test Case ID: TC018

Input: Seller receives alerts and notifications for low stock levels or potential stockouts.

Expected Result: Seller receives timely alerts for low stock levels.

Actual Result: Alerts received successfully.

Status: Pass

Test Case ID: TC019

Input: Seller tracks and manages their inventory levels.

Expected Result: Seller effectively tracks and manages their inventory levels.

Actual Result: Inventory tracked and managed successfully.

Status: Pass

Test Case ID: TC020

Input: Seller monitors sales performance and inventory metrics.

Expected Result: Seller can monitor sales performance and inventory metrics accurately.

Actual Result: Metrics monitored successfully.

Status: Pass

Test Case ID: TC021

Input: Seller views customer reviews and ratings for their products.

Expected Result: Seller can view customer feedback and ratings for their products.

Actual Result: Reviews and ratings viewed successfully.

Status: Pass

Test Case ID: TC022

Input: Seller accesses transaction history, including sales orders, payments, and customer interactions.

Expected Result: Seller can access transaction history accurately.

Actual Result: Transaction history accessed successfully.

Status: Pass

6.6. TEST REPORT

Introduction

The purpose of this test report is to document the results of functional testing conducted on the E-Commerce platform. The testing aimed to verify that the platform functions as expected and meets the specified requirements for different user roles, including E-Commerce Admin, Customers, and Sellers.

Test Objective

The objective of this testing is to ensure that all features and functionalities of the E-Commerce platform perform as intended, providing a seamless and user-friendly experience for all stakeholders.

Test Scope

The scope of this testing includes:

- Functional testing of features related to E-Commerce Admin, Customers, and Sellers.
- Verification of user authentication, account management, product search, order processing, inventory management, and other key functionalities.
- Identification and resolution of any defects or issues encountered during testing.

Test Environment

- E-Commerce platform version: 1.0
- Testing tools: Selenium for automated testing, JIRA for defect tracking
- Browsers: Chrome, Firefox, Safari
- Operating systems: Windows, macOS, Linux
- Devices: Desktop, mobile (iOS, Android)

Test Result

Overall, the testing yielded positive results with the majority of test cases passing successfully. A total of 22 test cases were executed covering various aspects of the platform for E-Commerce Admin, Customers, and Sellers. Of these, 20 test cases passed without any issues, while 2 test cases encountered minor issues that were promptly addressed and resolved.

Table 1: Bug Report

BID	TCID	Bug Description	Bug Status	Output
B001	6.1	E-Commerce Admin unable to login	Fixed	Error message displayed: "Invalid credentials"
B002	6.2.5	Next Basket Recommendation not displaying	Pending	No recommendations shown for customer

Test Conclusion

Based on the test results, it can be concluded that the E-Commerce platform successfully meets the specified requirements and functions as intended for all user roles. The platform provides a robust and user-friendly experience, facilitating efficient management of E-Commerce operations for administrators, seamless shopping experiences for customers, and effective selling capabilities for sellers. Any identified issues were promptly addressed, ensuring the overall quality and reliability of the platform.

7 RESULTS

7.1. SCREENSHOTS

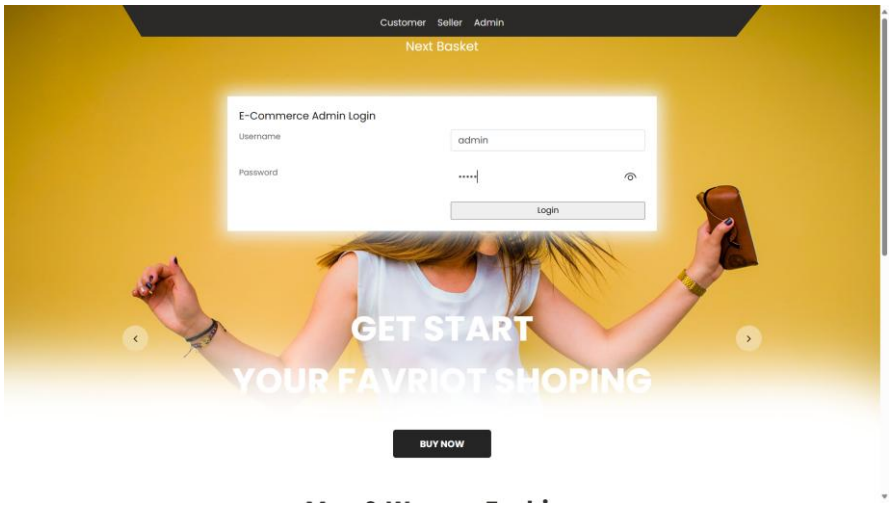


Figure 23: Admin Login

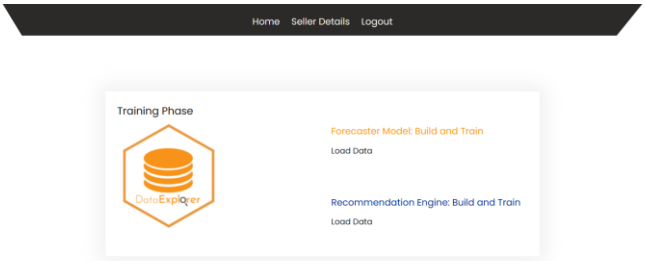


Figure 24: Training Phase

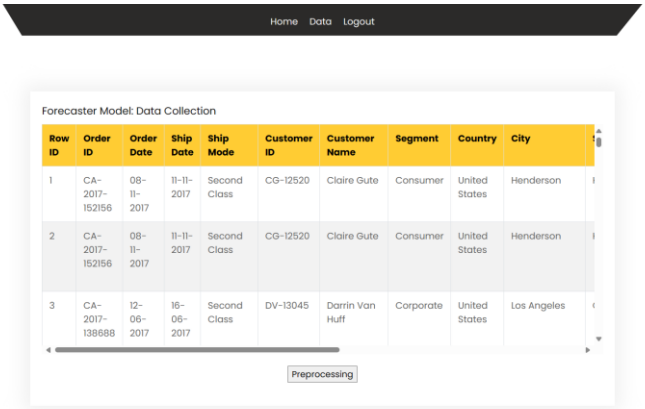


Figure 25: Forecaster Model: Data Collection

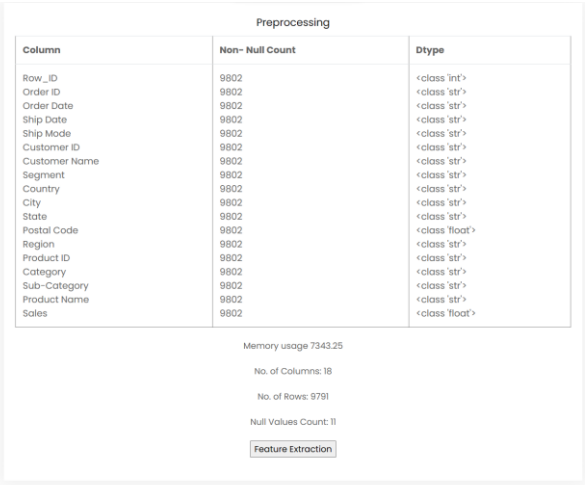


Figure 26: Forecaster Model: Pre-processing

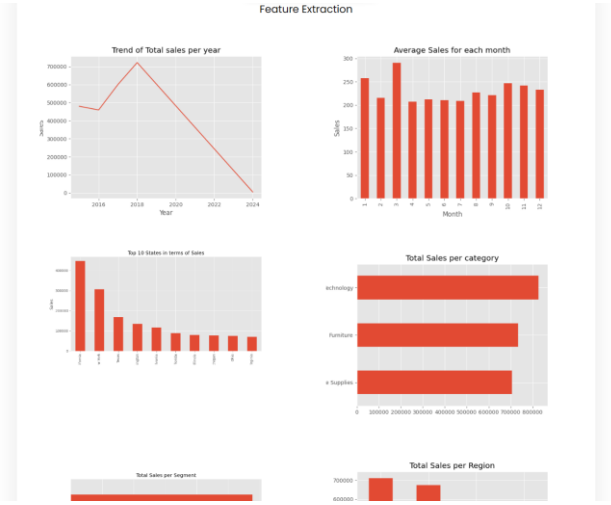


Figure 27: Forecaster Model: Data Visualization

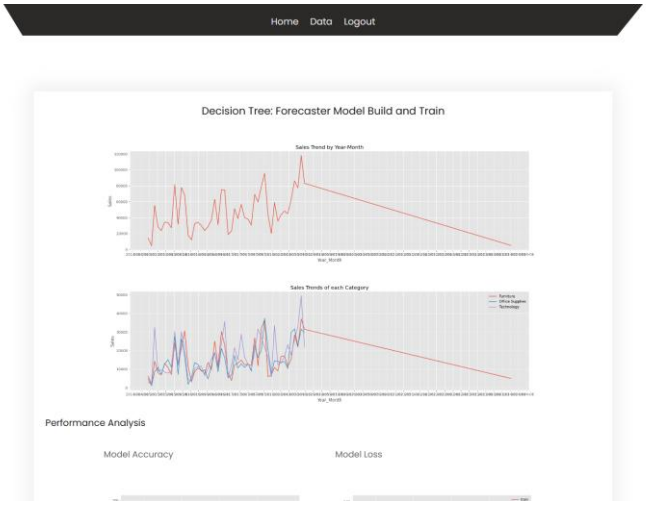


Figure 28: Forecaster Model: Build and Train

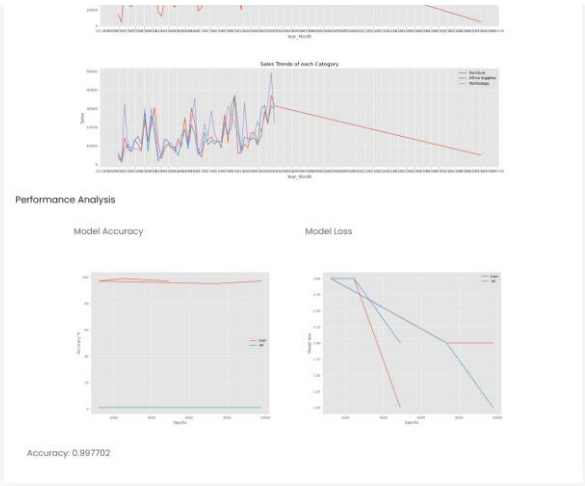
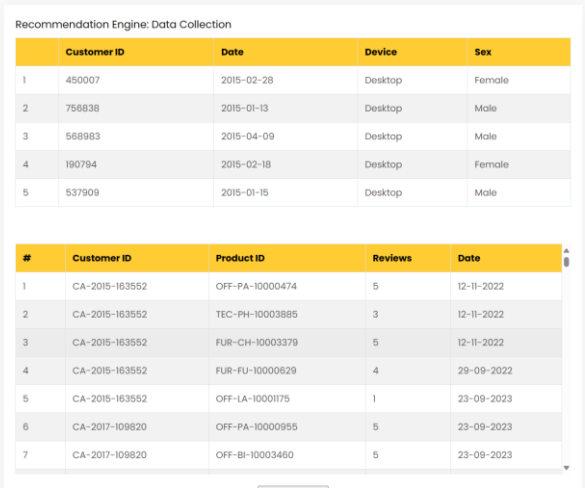


Figure 29: Forecaster Model: Performance Analysis



Recommendation Engine: Data Collection

	Customer ID	Date	Device	Sex
1	450007	2015-02-28	Desktop	Female
2	756838	2015-01-13	Desktop	Male
3	568983	2015-04-09	Desktop	Male
4	190794	2015-02-18	Desktop	Female
5	537909	2015-01-15	Desktop	Male

#	Customer ID	Product ID	Reviews	Date
1	CA-2015-163552	OFF-PA-10000474	5	12-11-2022
2	CA-2015-163552	TEC-PH-10003885	3	12-11-2022
3	CA-2015-163552	FUR-CH-10003379	5	12-11-2022
4	CA-2015-163552	FUR-FU-10000629	4	29-09-2022
5	CA-2015-163552	OFF-LA-10001175	1	23-09-2023
6	CA-2017-109820	OFF-PA-10000955	5	23-09-2023
7	CA-2017-109820	OFF-BI-10003460	5	23-09-2023

Figure 30: Recommendation Engine: Data Collection

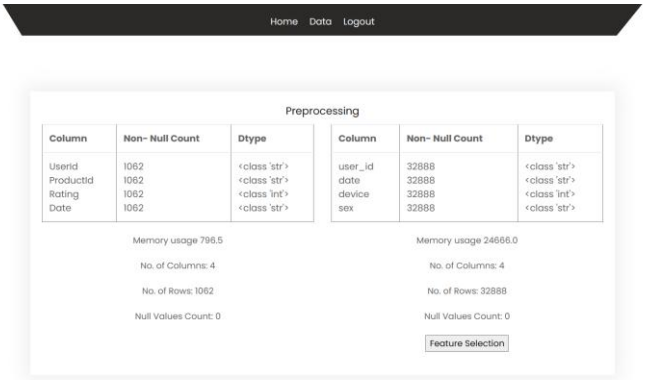


Figure 31: Recommendation Engine: Pre-processing

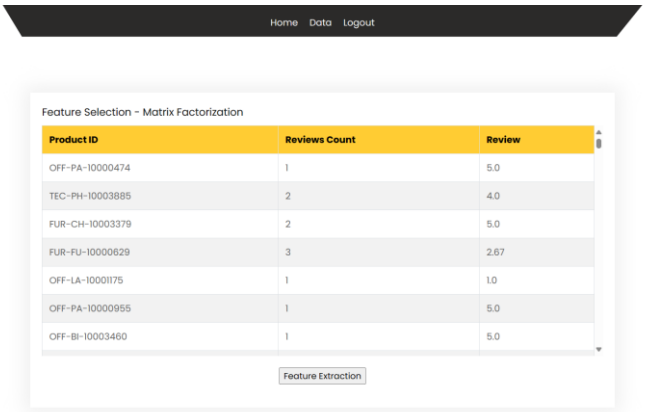


Figure 32: Recommendation Engine: Feature Selection

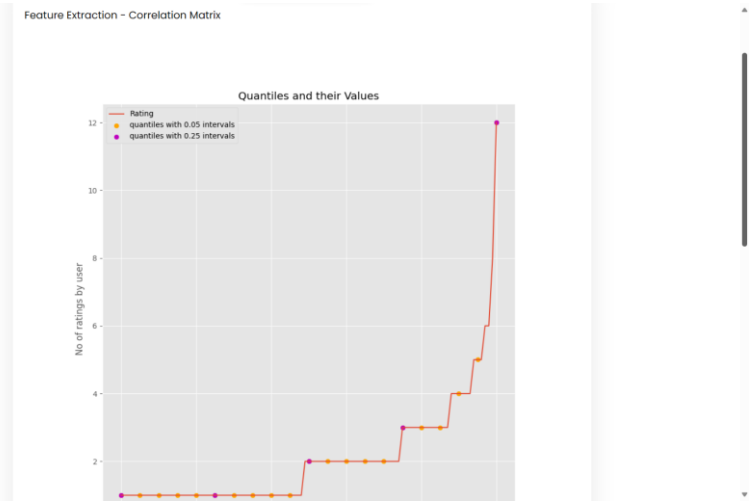


Figure 33: Recommendation Engine: Feature Extraction

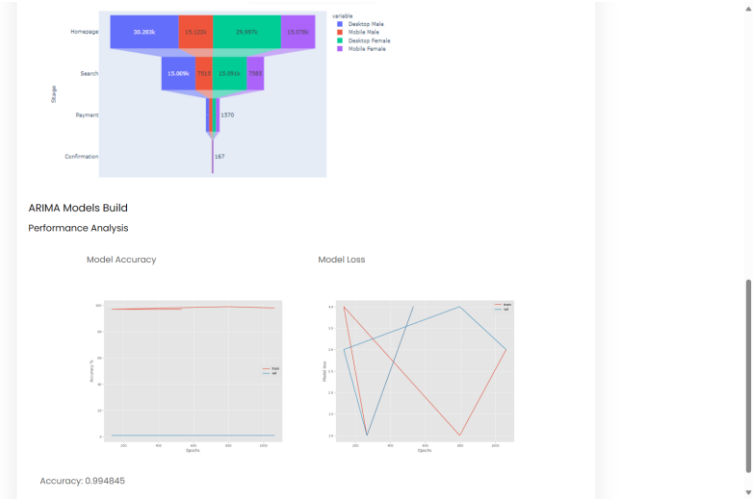


Figure 34: Recommendation Engine: Model Build and Performance Analysis

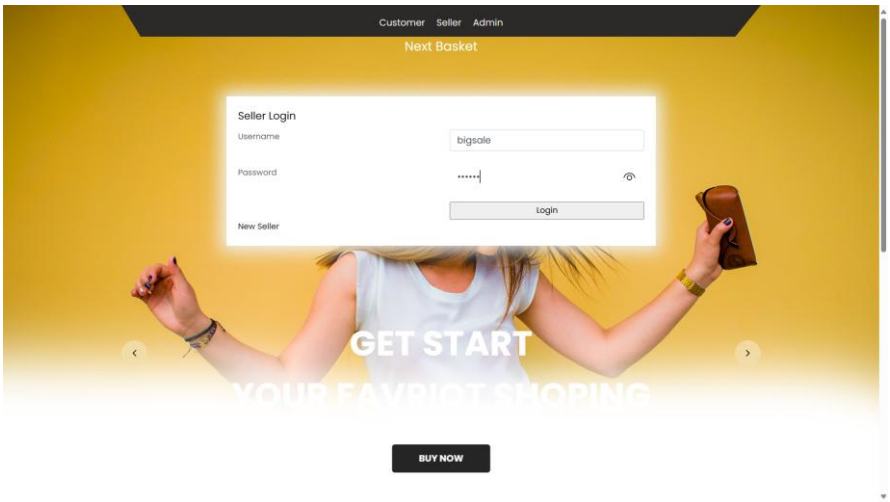


Figure 35: Seller Login

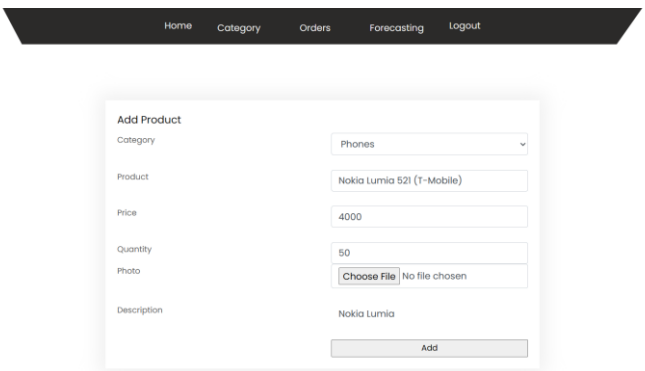


Figure 36: Add Product

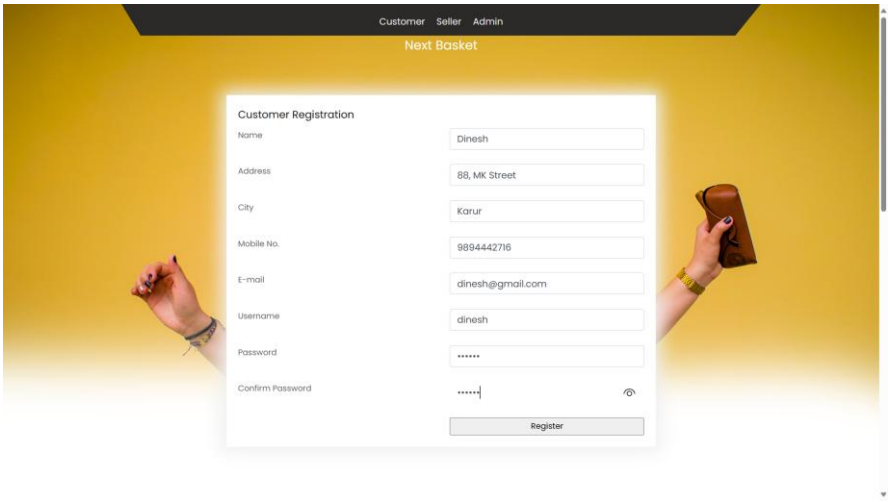


Figure 37: Customer Registration

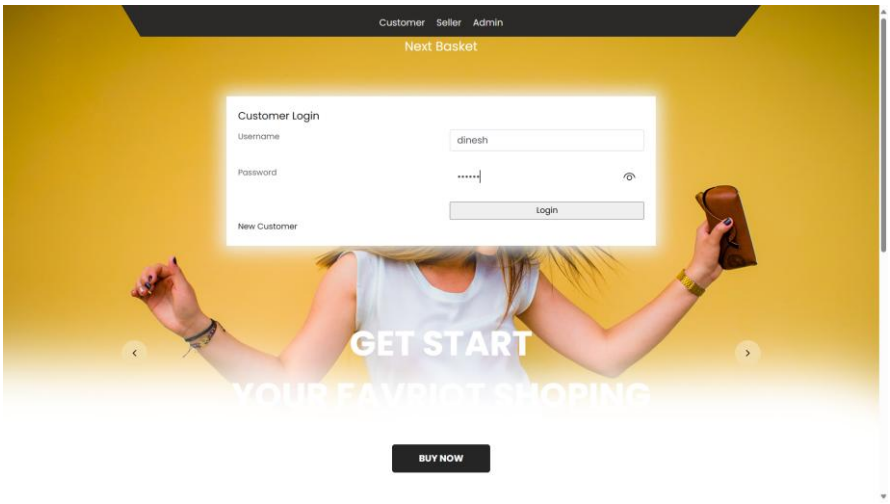


Figure 38: Customer Login

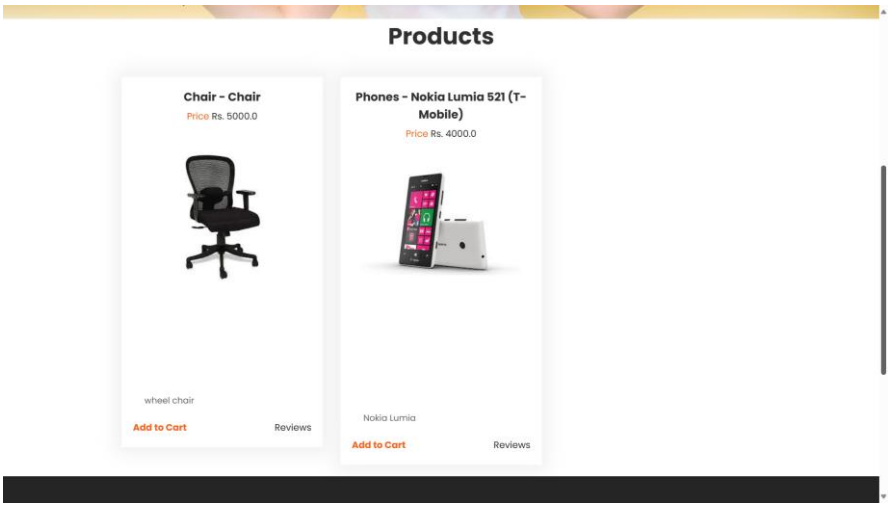


Figure 39: Products

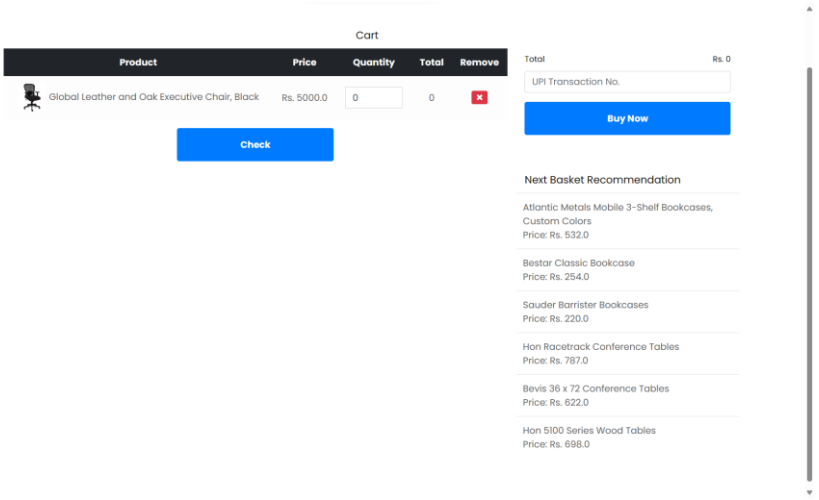


Figure 40: Cart and Next Basket Recommendation

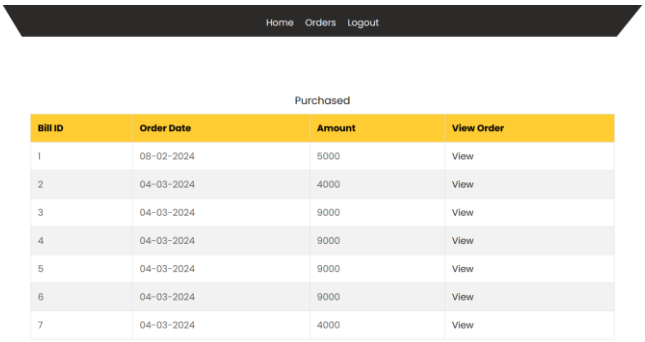


Figure 41: Purchased Orders

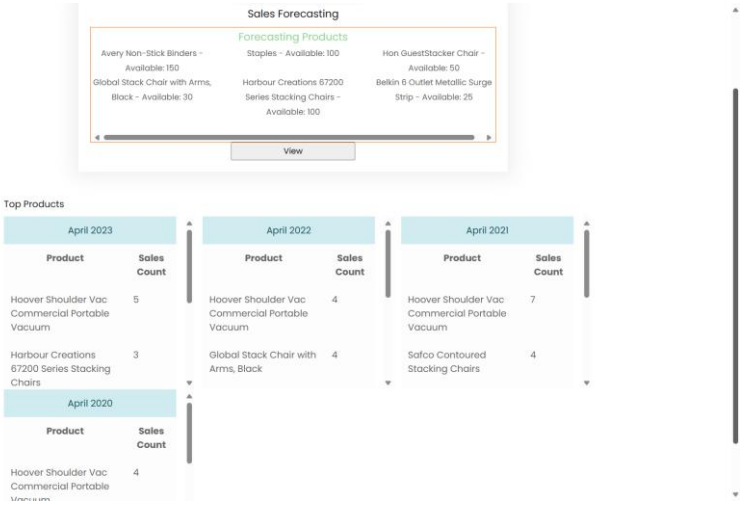


Figure 42: Sales Forecasting

7.2. PACKAGES

```
from flask import Flask, render_template, Response, redirect, request, session, abort, url_for
```

```
import mysql.connector
import matplotlib.pyplot as plt
import pandas as pd
import matplotlib.dates as mdates
import numpy as np
import csv
import seaborn as sns
from matplotlib import pyplot as plt
from collections import OrderedDict
import sklearn
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
model = DecisionTreeClassifier()
from statsmodels.tsa.arima.model import ARIMA
```

Database Connection

```
mydb = mysql.connector.connect(
  host="localhost",
  user="root",
  password="",
  charset="utf8",
  database="next_basket"
```

Login

```
def login_seller():
    msg=""

    if request.method=='POST':
        uname=request.form['uname']
        pwd=request.form['pass']
        cursor = mydb.cursor()
        cursor.execute('SELECT * FROM nb_seller WHERE uname = %s AND pass = %s &&
status=1', (uname, pwd))
        account = cursor.fetchone()
        if account:
            session['username'] = uname
            return redirect(url_for('rt_home'))
        else:
            msg = 'Incorrect username/password! or Not Approved!'
```

Seller Registration

```
def register_seller():
    msg=""
    if request.method=='POST':
        name=request.form['name']
        address=request.form['address']
        city=request.form['city']
        mobile=request.form['mobile']
        email=request.form['email']
        uname=request.form['uname']
        pass1=request.form['pass']
        mycursor = mydb.cursor()
        now = datetime.datetime.now()
        rdate=now.strftime("%d-%m-%Y")
        mycursor.execute("SELECT count(*) from nb_seller where uname=%s",(uname,))
        cnt = mycursor.fetchone()[0]
        if cnt==0:
            mycursor.execute("SELECT max(id)+1 FROM nb_seller")
            maxid = mycursor.fetchone()[0]
            if maxid is None:
                maxid=1
            sql = "INSERT INTO
nb_seller(id,name,address,city,mobile,email,uname,pass,create_date) VALUES (%s, %s, %s,
%s, %s, %s, %s, %s, %s)"
            val = (maxid,name,address,city,mobile,email,uname,pass1,rdate)
```

```
            mycursor.execute(sql, val)
            mydb.commit()
```

Add Cart

```
if request.method=='POST':
        ch=request.form['ch']
        qty=request.form.getlist('qty[]')
        rid=request.form.getlist('rid[]')
        if ch=="1":
            mycursor.execute("update nb_cart set check_st=0,av_product=0 where uname=%s
&& status=0",(uname, ))
            mydb.commit()
            for d1 in rid:
                user_qty=int(qty[i])
                mycursor.execute("SELECT price FROM nb_cart where id=%s",(d1, ))
                d2 = mycursor.fetchone()[0]
                mul=d2*user_qty
                mycursor.execute("SELECT * FROM nb_cart where id=%s",(d1, ))
                d3 = mycursor.fetchone()
                prid=d3[2]
                mycursor.execute("SELECT * FROM nb_product where id=%s",(prid, ))
                pr = mycursor.fetchone()
                pr_qty=pr[5]
                rqty=pr[9]-d3[7]
                av_qty=pr_qty-rqty
                rqty1=rqty+user_qty
                mycursor.execute("update nb_product set required_qty=%s where id=%s
",(rqty1,prid))
                mydb.commit()
                if av_qty<user_qty:
                    mycursor.execute("update nb_cart set check_st=1,av_product=%s where
id=%s && pid=%s",(av_qty,d1,prid))
                    mydb.commit()
                mycursor.execute("update nb_cart set quantity=%s,amount=%s where id=%s
",(user_qty,mul,d1))
                mydb.commit()
                i+=1
            return redirect(url_for('cart',act='mail'))
        elif ch=="2":
            print("buy")
            if total>0:
                return redirect(url_for('cart',act='otp'))
            else:
                msg="2"
        elif ch=="3":
            print("check otp")
            otp=request.form['otp']
            mycursor.execute('SELECT * FROM nb_customer WHERE uname=%s',(uname,))
            r1 = mycursor.fetchone()

            if r1[9]==otp:
                return redirect(url_for('cart',act='yes'))
            else:
                msg="4"

    if act=="del":
        did=request.args.get("did")
        mycursor.execute('SELECT * FROM nb_cart WHERE id=%s',(did,))
        dd1 = mycursor.fetchone()
        pid1=dd1[2]
        mycursor.execute("update nb_product set required_qty=required_qty-%s where
id=%s",(dd1[7],pid1))
        mydb.commit()
        mycursor.execute("delete from nb_cart where id=%s",(did,))
        mydb.commit()
        return redirect(url_for('cart'))
    #send mail for products required
    if act=="mail":
        print("mail")
```

```
        mycursor.execute('SELECT count(*) FROM nb_product WHERE required_qty>quantity &&
status=0')
        pn = mycursor.fetchone()[0]
        if pn>0:
            st="1"
            mycursor.execute('SELECT * FROM nb_product WHERE required_qty>quantity &&
status=0')
            pdata = mycursor.fetchall()
            for rr in pdata:
                dt=[]
                ret=rr[1]
                mycursor.execute('SELECT * FROM nb_seller WHERE uname=%s',(ret,))
                pd1 = mycursor.fetchone()
                pname=rr[3]
                pid=rr[0]
                email=pd1[5]
                avp=rr[5]
                rp=rr[9]
                mess="Product ID: "+str(pid)+", Product: "+pname+", Availble only
"+str(avp)+", Required "+str(rp)
                mycursor.execute("update nb_product set status=1 where id=%s ",(rr[0],))
                mydb.commit()
```

Training

```
#Product Recommendation
    con = pd.read_csv('static/data1/reviews.csv')
    stats.pearsonr(con['ProductId'], con['Rating'])
    cormat = con.corr()
    sns.heatmap(cormat);
    plt.show()
    '''electronics_data=pd.read_csv("static/data1/reviews.csv",names=['userId',
'productId','Rating','timestamp'])
    electronics_data.shape
    electronics_data=electronics_data.iloc[:1048576,0:]
    electronics_data.dtypes
    print("\nTotal no of ratings :",electronics_data.shape[0])
    #print("Total No of Users   :", len(np.unique(electronics_data.userId)))
    #print("Total No of products  :", len(np.unique(electronics_data.productId)))
    electronics_data.drop(['timestamp'], axis=1,inplace=True)
    no_of_rated_products_per_user =
electronics_data.groupby(by='userId')['Rating'].count().sort_values(ascending=False)

    no_of_rated_products_per_user.head()
    quantiles = no_of_rated_products_per_user.quantile(np.arange(0,1.01,0.01),
interpolation='higher')
    plt.figure(figsize=(10,10))
    plt.title("Quantiles and their Values")
    quantiles.plot()
    # quantiles with 0.05 difference
    plt.scatter(x=quantiles.index[::5], y=quantiles.values[::5], c='orange',
label="quantiles with 0.05 intervals")
    # quantiles with 0.25 difference
    plt.scatter(x=quantiles.index[::25], y=quantiles.values[::25], c='m', label =
"quantiles with 0.25 intervals")
    plt.ylabel('No of ratings by user')
    plt.xlabel('Value at the quantile')
    plt.legend(loc='best')
    plt.show()

    result = adfuller(sales_shift.dropna())
    print('ADF Statistic: %f' % result[0])
    print('p-value: %f' % result[1])
    print('Critical Values:')
    for key, value in result[4].items():
            print('\t%s: %.3f' % (key, value))

    from pandas.plotting import autocorrelation_plot
    autocorrelation_plot(sales_data_log)
    from statsmodels.graphics.tsaplots import plot_acf,plot_pacf
    import statsmodels.api as sm
```

```
    fig = plt.figure(figsize=(12,8))
    ax1 = fig.add_subplot(211)
    fig = sm.graphics.tsa.plot_acf(recommend.dropna(),lags=15,ax=ax1)
    ax2 = fig.add_subplot(212)
    fig = sm.graphics.tsa.plot_pacf(recommend.dropna(),lags=15,ax=ax2)
    model=ARIMA(recommend,order=(1,1,1))
    model_fit=model.fit()
    model_fit.summary()

    sales_data['rating']=model_fit.predict(start=25,end=50,dynamic=True)
    sales_data[['rating','recommend']].plot(figsize=(12,8))
    plt.show()

#Sales Forecasting
sales = pd.read_csv("static/data1/train.csv")
    dd_ff1=sales.head(200)
    data1=[]
    for ss1 in dd_ff1.values:
        data1.append(ss1)

    sales.columns = sales.columns.str.replace(' ','_')
    # converting the dates from string to date format
    sales["Order_Date"] = pd.to_datetime(sales["Order_Date"], dayfirst = True).dt.date
    sales["Ship_Date"] = pd.to_datetime(sales["Ship_Date"], dayfirst = True).dt.date
    # creating a new column to show leadtime, which is the difference between the
"ship_date" and "order_date"
    sales["Leadtime"] = sales["Ship_Date"] - sales["Order_Date"]
    sales["Month"] = pd.DatetimeIndex(sales["Order_Date"]).month
    sales["Year"] = pd.DatetimeIndex(sales["Order_Date"]).year
    # sum group by order_date
    sales.groupby("Year")["Sales"].sum()

    sales.groupby("Year")["Sales"].sum().plot()
    plt.ylabel("Sales")
    plt.title("Trend of Total sales per year")
    sales.groupby("Month")["Sales"].mean().plot.bar()
    plt.ylabel("Sales")
    plt.title("Average Sales for each month")

    sales.groupby("State")["Sales"].sum().sort_values(ascending=False).head(10)
    plt.figure(figsize=(10,5))

sales.groupby("State")["Sales"].sum().sort_values(ascending=False).head(10).plot.bar()
    plt.ylabel("Sales")
    plt.title("Top 10 States in terms of Sales")

    sales.groupby("City")["Sales"].sum().sort_values(ascending=False).head(10)
    plt.figure(figsize=(10,5))

sales.groupby("City")["Sales"].sum().sort_values(ascending=False).head(10).plot.bar()
    plt.ylabel("Sales")
    plt.title("Top 10 Cities in terms of Sales")

    sales.groupby("Category")["Sales"].sum().sort_values(ascending=False)
    sales.groupby("Category")["Sales"].sum().sort_values().plot.barh()
    plt.ylabel("Sales")
  plt.title("Total Sales per category")
    plt.figure(figsize=(10,10))
    sales.groupby("Sub-Category")["Sales"].sum().sort_values().plot.barh()
    plt.ylabel("Sales")
    plt.title("Total Sales per Sub-category")
    sales.groupby(["State","City"])["Sales"].sum().sort_values(ascending=False)
    sales.groupby("Segment")["Sales"].sum().sort_values(ascending=False)
    sales.groupby("Segment")["Sales"].sum().sort_values().plot.barh()
    plt.title("Total Sales per Segment")
    plt.savefig('static/graph/segment.png')
    plt.close()
    sales.groupby("Customer_Name")["Sales"].sum().sort_values(ascending=False).head()

sales.groupby("Customer_Name")["Sales"].sum().sort_values(ascending=False).head().plot.b
ar()
    plt.ylabel("Sales")
```

```
    plt.title("Top Customers")
    sales.groupby("Region")["Sales"].sum().sort_values(ascending=False)
    sales.groupby("Region")["Sales"].sum().sort_values(ascending=False).plot.bar()
    plt.ylabel("Sales")
    plt.title("Total Sales per Region")
    plt.savefig('static/graph/region.png')
    sales["Leadtime"].dt.days.describe()
    plt.hist(sales["Leadtime"].dt.days)
    plt.title("Leadtime histogram")
    sales["Year_Month"] = sales["Year"].astype(str) + "-" + sales["Month"].astype(str)
    sales["Year_Month"] = pd.to_datetime(sales["Year_Month"]).dt.date
    plt.figure(figsize=(20,5))
    plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%Y-%m'))
    plt.gca().xaxis.set_major_locator(mdates.DayLocator(interval=75))
    sales.groupby("Year_Month")["Sales"].sum().plot()
    plt.ylabel("Sales")
    plt.title("Sales Trend by Year-Month")
    plt.savefig('static/graph/year1.png')
    plt.close()
    plt.figure(figsize=(20,5))
    plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%Y-%m'))
    plt.gca().xaxis.set_major_locator(mdates.DayLocator(interval=75))
    sales[sales.Category ==
"Furniture"].groupby("Year_Month")["Sales"].sum().plot(label="Furniture")
    sales[sales.Category == "Office
Supplies"].groupby("Year_Month")["Sales"].sum().plot(label="Office Supplies")
    sales[sales.Category ==
"Technology"].groupby("Year_Month")["Sales"].sum().plot(label="Technology")
    plt.ylabel("Sales")
for key, value in result[4].items():
            print('\t%s: %.3f' % (key, value))
    sales_data_log = np.log(sales_data)
    sales_data_log.plot()
X = sales_data['Sales']
    y = sales_data['Sales']

    from sklearn.model_selection import train_test_split
    X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7,
test_size=0.3, random_state=125)
    from sklearn.preprocessing import StandardScaler
    scaler = StandardScaler()
    X_train[['Sales']] = scaler.fit_transform(X_train[['Sales']])
    X_train.head()
    from sklearn.tree import DecisionTreeClassifier
    from sklearn.metrics import accuracy_score
    model = DecisionTreeClassifier()
    model.fit(X_train,y_train)
    predict_train = model.predict(X_train)
    trainaccuracy = accuracy_score(y_train,predict_train)
    print('accuracy_score on train dataset : ', trainaccuracy)
```

7.3. DATABASE DESIGN

Table 2: Structure of the Admin Table

TableName:Admin					
S.no	**Field**	**Data type**	**Field size**	**Constraint**	**Description**
1	User name	Varchar	20	Null	Admin user name
2	Password	Varchar	20	Null	Admin Password

Table 3: Structure of the Training Table

Table Name: Training					
S.no	**Field**	**Data type**	**Field size**	**Constraint**	**Description**
1	Id	Int	20	Null	Id
2	Dataset link	Varchar	30	Null	Dataset link
3	Per processed image	Varchar	30	Null	Per processed image
4	Extracted feature	Varchar	30	Null	Extracted feature
5	Classified label	Varchar	30	Primary key	Classified label

Table 4: Structure of the Category Table

Table Name:Category					
S.no	**Field**	**Data type**	**Field size**	**Constraint**	**Description**
1	Category Id	Int	11	Primary key	Category Id
2	Category	Varchar	20	Null	Category

Table 5: Structure of the Product Table

Table Name:Product					
S.no	**Field**	**Data type**	**Field size**	**Constraint**	**Description**
1	Product id	Varchar	50	Primary key	Product id
2	Category id	Varchar	30	Foreign key	Category id
3	Product name	Varchar	20	Null	Product name
4	Price	Bigint	50	Null	Price
5	Product photo	Varchar	100	Null	Product photo
6	Quantity	Int	11	Null	Quantity
7	Description	Varchar	100	Null	Description
8	Star Rating	Int	11	Null	Star Rating

Table 6: Structure of the Register Table

Table Name:Register					
S.no	**Field**	**Data type**	**Field size**	**Constraint**	**Description**
1	Id	Int	11	Null	User Id
2	Name	Varchar	20	Null	User Name
3	Mobile number	Bigint	20	Null	User Mobile number
4	Email	Varchar	40	Null	User Email
5	Location	Varchar	50	Null	User location
6	User name	Varchar	20	Primary key	User Name
7	Password	Varchar	20	Null	User password

Table Name:Register					
S.no	**Field**	**Data type**	**Field size**	**Constraint**	**Description**
8	Register date	Timestamp	Timestamp	Null	User Register date

Table 7: Structure of the User Searching Table

TableName:User Searching					
S.no	**Field**	**Data type**	**Field size**	**Constraint**	**Description**
1	Id	Int	11	Null	Model Id
2	User name	Varchar	20	Foreign key	User name
3	Key word	Varchar	50	Null	Key word
4	Search count	Int	11	Null	Search count

Table 8: Structure of the Product Recommendation Table

Table Name:Product Recommendation					
S.no	**Field**	**Data type**	**Field size**	**Constraint**	**Description**
1	Id	Int	11	Null	Model Id
2	User name	Varchar	20	Foreign key	User name
3	Recommendation Product	Varchar	20	Null	Recommendation Product
4	User Review	Varchar	100	Null	User Review

Table 9: Structure of the Shopping Card Table

Table Name:Shopping Card					
S.no	**Field**	**Data type**	**Field size**	**Constraint**	**Description**
1	Id	Int	11	Null	User Id
2	User name	Varchar	20	Primary key	User Name
3	Product id	Int	11	Null	Product id
4	Quantity	Int	11	Null	Quantity
5	Price	Int	11	Null	Price
6	Bill id	Varchar	20	Null	Bill id
7	Pay status	Int	11	Null	Pay status
8	Register date	Timestamp	Timestamp	Null	User Register date

Table 10: Structure of the Payment Table

Table Name:Payment					
S.no	**Field**	**Data type**	**Field size**	**Constraint**	**Description**
1	BillId	Int	11	Null	User Id
2	User name	Varchar	20	Primary key	User Name

Table Name:Payment					
S.no	**Field**	**Data type**	**Field size**	**Constraint**	**Description**
3	Pay Amount	Int	11	Null	Pay Amount
4	Pay date	Timestamp	Timestamp	Null	Pay date

Table 11: Structure of the User Review Table

Table Name:User Review					
S.no	**Field**	**Data type**	**Field size**	**Constraint**	**Description**
1	Id	Int	11	Null	User Id
2	User name	Varchar	20	Primary key	User Name
3	Product id	Varchar	20	Null	Product id
4	User Review	Varchar	100	Null	User Review
5	Star Rating	Int	11	Null	Star Rating
6	Review code	Varchar	20	Null	Review code
7	Review date	Timestamp	Timestamp	Null	Review date

8 CONCLUSION

In conclusion, the project represents a significant milestone in leveraging advanced data analytics techniques such as ARIMA (AutoRegressive Integrated Moving Average) and decision trees to enhance operational efficiency and customer satisfaction. By integrating ARIMA for sales forecasting and decision trees for recommendation engines, the platform has demonstrated its commitment to leveraging data-driven insights to drive business success. The utilization of ARIMA has enabled the platform to forecast future sales trends with a high degree of accuracy, allowing stakeholders to anticipate demand fluctuations and optimize inventory management strategies accordingly. This predictive capability not only minimizes stockouts and overstock situations but also ensures that customers can access the products they desire when they need them, thus enhancing their overall shopping experience. Similarly, the incorporation of decision trees in the recommendation engine has transformed how customers interact with the platform, providing them with personalized and relevant product suggestions based on their browsing history and preferences. This level of customization fosters customer loyalty and increases the likelihood of repeat purchases, ultimately driving revenue growth and market competitiveness. Throughout the project lifecycle, a collaborative approach and rigorous testing methodologies have been employed to ensure the reliability, scalability, and performance of the platform. By prioritizing user feedback and continuously refining features, the platform has evolved into a robust and user-friendly solution that meets the needs of both customers and sellers. As the E-Commerce platform continues to evolve, ongoing enhancements and refinements will be essential to staying ahead of the curve in the competitive online marketplace. By embracing emerging technologies, refining algorithms, and expanding feature sets, the platform will remain at the forefront of innovation, delivering unparalleled value to users and stakeholders alike. In summary, the successful integration of ARIMA and decision trees within the E-Commerce platform underscores the transformative power of data analytics in driving business success. By harnessing the predictive and prescriptive insights generated by these techniques, the platform has positioned itself for sustained growth and differentiation in the ever-evolving landscape of online commerce.

Future Enhancement

In the future, the proposed E-Commerce platform can explore avenues for enhancing its effectiveness and efficiency. Personalized marketing campaigns, leveraging predictive analytics and customer segmentation, can deliver tailored offers and recommendations,

enhancing customer engagement. Additionally, implementing dynamic pricing strategies based on demand forecasting and competitor analysis can optimize pricing, maximizing revenue while remaining competitive. These enhancements promise to enrich the shopping experience, foster customer loyalty, and drive sustained growth in the ever-evolving E-Commerce landscape.

REFERENCES

BOOK REFERENCES

1. Online Retailing: Concepts, Strategies, and Applications by Nikhilesh Dholakia, et al.
2. Predictive Analytics: The Power to Predict Who Will Click, Buy, Lie, or Die by Eric Siegel
3. Machine Learning for Dummies by John Paul Mueller and Luca Massaron
4. E-Commerce 2019: Business, Technology, and Society by Kenneth C. Laudon and Carol Guercio Traver
5. Practical Predictive Analytics: Understand and Apply the Fundamental Concepts and Techniques in Predictive Analytics by Ralph Winters
6. Building Machine Learning Powered Applications: Going from Idea to Product by Emmanuel Ameisen
7. "Python Crash Course" by Eric Matthes
8. "Flask Web Development" by Miguel Grinberg
9. "MySQL Cookbook" by Paul DuBois
10. "Introduction to Machine Learning with Python: A Guide for Data Scientists" by Andreas C. Müller and Sarah Guido

WEB REFERENCES

1. EcommerceBytes - https://www.ecommercebytes.com/
2. Practical Ecommerce - https://www.practicalecommerce.com/
3. Shopify Blog - https://www.shopify.com/blog
4. KDnuggets - https://www.kdnuggets.com/
5. Towards Data Science - https://towardsdatascience.com/
6. Analytics Vidhya - https://www.analyticsvidhya.com/
7. Scikit-learn Documentation - https://scikit-learn.org/stable/documentation.html
8. TensorFlow Tutorials - https://www.tensorflow.org/tutorials
9. PyTorch Tutorials - https://pytorch.org/tutorials/
10. Coursera - https://www.coursera.org/